

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

tidyfinance: Transparent Factor Construction for Empirical Asset Pricing

Christoph Frey
Lancaster University

Christoph Scheuch
Humboldt University of Berlin

Stefan Voigt
University of Copenhagen

Patrick Weiss
Reykjavík University

Abstract

The R package **tidyfinance** provides research infrastructure for the empirical asset pricing data workflow. This workflow turns raw accounting and return data into factor portfolios for asset pricing tests, optimal portfolio allocation, and the design of trading strategies. **tidyfinance** exposes every construction decision as a configurable parameter with a documented default. A unified `download_data()` interface covers WRDS (CRSP, Compustat), Ken French's Data Library, FRED, and other sources, and applies provider-specific cleaning. Modular functions automate fiscal-year lagging, breakpoint computation, and portfolio aggregation. We illustrate the package by reproducing the asset growth factor end-to-end from raw data. Researchers who want precomputed series instead of rebuilding the pipeline can download a factor library of 50 sorting variables across 841,536 specifications with a single `download_data()` call or browse the time series at factors.tidyfinance.org.

Keywords: financial risk factors, reproducible research, asset pricing, R.

1. Introduction

Factor returns enter empirical asset pricing tests, optimal portfolio allocation, and risk-management decisions. Producing factor returns is a three-stage pipeline: data acquisition (e.g., downloading and cleaning raw accounting and return data from the Center for Research in Security Prices (CRSP), Compustat, and similar providers); variable construction (lagging variables to avoid look-ahead bias); and portfolio sorting (assigning securities to portfolios by characteristic). Each stage embeds construction choices: how to handle delistings, which

fiscal-year lag to apply, where to set breakpoints, and whether to equal- or value-weight portfolios. Few pipelines are standardized, and fewer still ship with executable code. Therefore, reproducing or extending a factor series often starts with reconstructing the pipeline from scratch, with no way to verify that the result matches the original approach.

No widely adopted tool covers this upstream workflow. The most popular factor sources publish *series* without the code that produced them: Ken French's Data Library (Fama and French 2023) supplies the Fama-French three- and five-factor returns and dozens of sorted portfolios. Some other projects release code: the Open Source Bond Asset Pricing project (Dickerson, Mueller, and Robotti 2023), available at openbondassetpricing.com, covers fixed income; the Open Source Cross-Sectional Asset Pricing repository (Chen and Zimmermann 2022) reproduces 213+ published anomalies, and Jensen, Kelly, and Pedersen (2023) release a global factor library spanning hundreds of long-short signals across many countries. None of these sources, however, exposes the preprocessing pipeline as configurable code that researchers can modify. Many individual authors also host their own published factors, with transparency about construction choices varying widely. Crucially, Walter, Weber, and Weiss (2024) and Soebhag, Van Vliet, and Verwijmeren (2024) show that these choices significantly affect the outcome of portfolio sorts. Existing R packages for quantitative finance operate downstream of factor construction, i.e., on price and return panels that the user must already have. Between raw data and standard asset-pricing tests sits an opaque layer, where construction choices accumulate silently.

This paper introduces **tidyfinance**, an R package on CRAN that exposes each stage of the pipeline as configurable code. The package addresses each stage with a dedicated interface. A unified `download_data()` function cleans, aligns, and harmonizes panels from WRDS (CRSP, Compustat, TRACE, FISD), Ken French's Data Library, FRED, and seven other providers, including current constituents for 20 major equity indexes; Table 2 catalogs the full set. Composable helpers then automate fiscal-year lagging, breakpoint computation, and portfolio aggregation. Every construction choice is exposed as a function argument with a default that follows established convention (for instance, value-weights and the Fama-French fiscal-year lag). A default call reproduces standard practice; overriding any default takes one argument.

We illustrate the package by reproducing a version of the asset growth factor of Cooper, Gulen, and Schill (2008) end-to-end from raw CRSP and Compustat data, then varying any construction choice with a single-argument change. To spare researchers the work of rebuilding the pipeline, we run **tidyfinance** across many defensible combinations and publish the results as a factor library of 50 sorting variables across 841,536 specifications. The library is available through a single `download_data()` call or an interactive dashboard at factors.tidy-finance.org. The workflow that the package operationalizes is developed in the open-access textbooks *Tidy Finance with R* (Scheuch, Voigt, and Weiss 2023) and *Tidy Finance with Python* (Scheuch, Voigt, Weiss, and Frey 2024), which present each step in tutorial form with inline scripts. The contribution of this paper is the **tidyfinance** CRAN package itself: a unified, configurable, tested, and versioned R API for that workflow. Where the books teach a reader how to assemble the pipeline from scratch, the package supplies it as a single dependency that researchers can call, audit, and reuse without rebuilding the underlying code.

2. Background and related work

The R ecosystem for empirical asset pricing splits cleanly into three groups. Each addresses one slice of the workflow and leaves the rest to the user.

Data-acquisition packages. Several R packages retrieve specific data sources commonly used in asset pricing. **frenchdata** (Areal 2021) downloads and parses the zip archives from Ken French's Data Library into long data frames; it does not touch WRDS and does not implement any construction step beyond reading the published factor series. **quantmod** (Ryan and Ulrich 2025) and **tidyquant** (Dancho and Vaughan 2026) cover Yahoo Finance, FRED, and Tiingo, returning daily open, high, low, and close price series and a small set of derived return helpers; neither connects to WRDS, and neither covers factor-construction logic. **Rblpapi** (Armstrong, Eddelbuettel, Laing, and Ooms 2024) queries the Bloomberg terminal for subscribers; the returned series leave the same factor-construction gap unfilled as the packages above. None of these packages covers the WRDS-CRSP-Compustat panel or the cleaning, alignment, and survivorship conventions the asset-pricing literature requires.

Downstream analysis packages. Once a cleaned panel is in hand, several packages support analysis. **PerformanceAnalytics** (Peterson and Carl 2026) computes performance and risk metrics for return series. **fPortfolio** (Wuertz, Setz, Chalabi, and Chen 2023) covers portfolio optimization. **RQuantLib** (Eddelbuettel, Nguyen, and Leitch 2024) interfaces with QuantLib for derivative pricing. **highfrequency** (Boudt, Cornelissen, Payseur, Kleen, and Sjoerup 2024) cleans intraday trade-and-quote panels and implements microstructure-volatility estimators; it operates on a panel the user must already have. Each of these tools assumes that the panel of returns and factors is already constructed; none implements a factor-construction layer.

Factor data releases. A group of projects publishes *factor return series* directly. Ken French's Data Library (Fama and French 2023) supplies the canonical Fama-French three- and five-factor returns and dozens of sorted portfolios, but documents construction in prose only and releases no construction code. Akey, Robertson, and Simutin (2026) show that these series change retroactively as the underlying method is silently revised, so that the factor returns a researcher obtains depend on *when* they were downloaded, with material effects on mutual-fund performance and cross-sectional asset pricing tests. A transparent, code-first pipeline removes precisely this opacity. Three more recent projects pair the published factor data with the construction script that produced it. Hou, Xue, and Zhang (2020) replicate 452 cross-sectional anomalies and document that most fail out-of-sample, attributing the failures in part to undocumented construction variation across studies. The Open Source Cross-Sectional Asset Pricing repository (Chen and Zimmermann 2022) responds by publishing per-anomaly construction scripts alongside the resulting 213+ long-short returns. The Open Source Bond Asset Pricing project (Dickerson *et al.* 2023) does the same for corporate bond factors. Jensen *et al.* (2023) release the Global Factor Data of hundreds of long-short signals across many countries, together with the construction code in Python. These projects represent a substantial move toward transparency: a reader can inspect exactly how each factor was built and reproduce its returns. Two limitations remain. First, none exposes the pipeline as a callable API. A user who wants the asset growth factor rebalanced in January instead of July, or formed on NYSE-plus-NASDAQ breakpoints, or computed on a different fiscal-year lag, must fork the published script and edit it by hand. The script is not a function with named arguments. Second, the factor returns are frozen at the release date, with no in-workflow path to refresh them as new observations arrive or as a researcher's design choices evolve.

Tidy finance: transparency and modularity. tidyfinance fills the missing middle: it

supplies the cleaned WRDS-CRSP-Compustat panel that data-acquisition packages do not produce and exposes the factor-construction pipeline that downstream analysis packages assume to be in hand. One `download_data()` call returns a cleaned WRDS-CRSP-Compustat panel; further functions implement the full factor-construction pipeline (fiscal-year lagging, breakpoint computation, portfolio sorting, long-short aggregation) as a chain of composable steps whose arguments cover the construction choices that vary across studies in the literature above. The companion textbooks *Tidy Finance with R* (Scheuch *et al.* 2023) and *Tidy Finance with Python* (Scheuch *et al.* 2024) develop the workflow itself in tutorial form, with the construction steps written out as inline scripts for readers to follow. The package is the software artifact that turns that workflow into a CRAN library: tested, versioned, and callable as a single dependency rather than copied chunk by chunk into each new project. Two design principles, both inherited from the textbooks, guide the API. *Transparency*: every construction decision is exposed as a configurable parameter with a documented default, following the *tidy data* discipline of Wickham (2014). *Modularity*: each construction step is a small function, so researchers can chain the full pipeline or call individual pieces inside their own workflows. **tidyfinance** additionally covers asset pricing tests and optimal portfolio choice; this paper focuses on factor construction and defers the remaining functionality to dedicated treatments.

3. Package overview

3.1. Installation and setup

You can install the released version of **tidyfinance** from CRAN via:

```
R> install.packages("tidyfinance")
```

To install the development version of **tidyfinance** from GitHub, first install **pak**, then run:

```
R> install.packages("pak")
R> pak::pak("tidy-finance/r-tidyfinance")
```

Once installed, load the package with:

```
R> library("tidyfinance")
```

3.2. Design and workflow

The empirical asset pricing workflow decomposes into three stages: data acquisition, variable construction, and portfolio sorting. Each stage involves dozens of small decisions: universe filters, cleaning rules, rebalancing conventions, and breakpoint definitions. Table 1 catalogs the user-facing functions exported by **tidyfinance**, grouped by workflow stage. The remainder of this section walks through them in narrative order.

Function	Purpose
Data acquisition	
`download_data()`	Retrieves and cleans a panel from supported data providers.

Function	Purpose
`set_wrds_credentials()`	Stores WRDS username and password in `.Renviron` for subsequent calls.
Variable construction	
`add_lagged_columns()`	Attaches the most recent observed value within a configurable look-back window, with optional Fama-French timing.
`join_lagged_values()`	Merges lagged values from a source panel into a target panel using the same windowing logic.
`winsorize()`	Replaces values outside the q-th and $(1-q)$-th percentiles with the corresponding quantile.
`trim()`	Sets values outside the q-th and $(1-q)$-th percentiles to `NA`.
`compute_rolling_value()`	Applies an aggregation function over a calendar-time rolling window.
`filter_sorting_data()`	Applies the universe screens in `filter_options()`, such as excluding financials, utilities, or low-priced or small stocks.
Portfolio sorting	
`compute_breakpoints()`	Computes quantile cutoffs from a cross-section, optionally restricted by exchange or by minimum market-cap quantile.
`assign_portfolio()`	Maps each observation to a portfolio integer using supplied or computed breakpoints.
`compute_portfolio_returns()`	Forms portfolios period by period and returns equal-, value-, and capped-value-weighted return panels.
`compute_long_short_returns()`	Computes the top-minus-bottom (or bottom-minus-top) spread from the output of `compute_portfolio_returns()`.
`implement_portfolio_sort()`	Wraps filtering, breakpoint computation, assignment, and aggregation into a single call for univariate and bivariate sorts.
Asset pricing tests	
`estimate_fama_macbeth()`	Runs cross-sectional ordinary least squares (OLS) regressions each period and aggregates into average risk premia with Newey-West standard errors (Newey and West 1987).
`estimate_betas()`	Estimates rolling-window market betas for each asset.
`estimate_model()`	Fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame.

Table 1: User-facing functions in **tidyfinance**, grouped by workflow stage. Functions whose first argument is a panel chain naturally via the native pipe `|>`; `filter_options()`, `breakpoint_options()`, `portfolio_sort_options()`, and `data_options()` (explained in detail below) bundle the configurable arguments for the sorting helpers.

3.3. Data acquisition

tidyfinance exposes one interface, `download_data()`, to 326 datasets: equity returns, fixed-income transactions, macroeconomic predictors, index constituents, and more. Table 2 lists all available sources. `list_supported_datasets()` returns the full catalog programmatically. The companion Python package **tidyfinance**, hosted on PyPI, exposes the same `download_data()` interface, with identical signatures and dataset identifiers, so analyses can be ported between the two languages without changes to the data layer.

Source	Description	Example
Public data sources		

Ken French's Data Library	Fama-French 169+ factor models, portfolios, industry classifications	`download_data(`
		`  domain = "factors_ff",`
		`  dataset =`
		`  "Fama/French 3 Factors")`

Global Q	Hou, Mo, Xue, and Zhang (2021) q5 factor model (market, size, investment, ROE, expected growth) at multiple frequencies	`download_data(`
		`  domain = "globalq",`
		`  dataset =`
		`  "factors_q5_monthly")`

Goyal-Welch Predictors	Macroeconomic return predictors	`download_data(`
		`  domain = "macro_predictors",`
		`  dataset = "monthly")`

FRED	St. Louis Fed economic time series (GDP, CPI, etc.)	`download_data(`
		`  domain = "fred",`
		`  series = "GDP")`

Yahoo Finance	Historical price data with adjusted close for any ticker	`download_data(`
		`  domain = "stock_prices",`
		`  symbols = "AAPL")`

Index Constituents	Current members of major equity indexes (S&P 500, Russell, DAX, MSCI World, etc.)	`download_data(`
		`  domain = "constituents",`
		`  index = "DAX")`

Open Source Asset Pricing	213+ monthly asset pricing signals and anomalies	`download_data(` `  domain = "osap")`

High-Frequency	5-second frequency data for S&P 500 constituents	`download_data(`
		`  domain = "tidyfinance",`
		`  dataset =`
		`  "high_frequency_sp500")`

Risk-Free Rate	Monthly risk-free rate (FRED T-bill series)	`download_data(`
		`  domain = "tidyfinance",`
		`  dataset = "risk_free")`

Factor Library	Precomputed factor returns across construction choices	`download_data(`
		`  domain = "tidyfinance",`
		`  dataset = "factor_library",`
		`  sorting_variable = "ag")`

WRDS (subscription required)		

Source	Description	Example

CRSP	Daily and monthly stock returns, market cap, listing exchange, industry information	`download_data(`
		`  domain = "wrds",`
		`  dataset = "crsp_monthly")`

Compustat	Annual and quarterly financial statements	`download_data(`
		`  domain = "wrds",`
		`  dataset = "compustat_annual")`

CCM Links	CRSP-Compustat linking table (permno to gvkey)	`download_data(`
		`  domain = "wrds",`
		`  dataset = "ccm_links")`

FISD	Mergent Fixed Income Securities Database	`download_data(`
		`  domain = "wrds",`
		`  dataset = "fisd")`

TRACE	Corporate bond transactions (Dick-Nielsen (2009) cleaned)	`download_data(`
		`  domain = "wrds",`
		`  dataset = "trace_enhanced",`
		`  cusips = "655844CJ5")`

Table 2: Data sources available via `download_data()` in **tidyfinance**. For every dataset with a time dimension, the same `start_date` and `end_date` arguments subset the returned panel to a user-specified window.

Most rows in Table 2 mirror upstream providers; the high-frequency dataset is the exception, which we host ourselves. It comprises message-level snapshots of the NASDAQ limit order book for the SPY exchange-traded fund (and TLT for government bonds) from July 2007 to October 2025, more than 30 billion order-book messages in total, curated for the analysis in Hautsch, Menkveld, and Voigt (2026).

A first illustration uses only public data sources. We retrieve the current constituents of the Dow Jones Industrial Average and then download daily prices for those tickers over a chosen window. `list_supported_indexes()` returns the full set of equity indexes available through the `"constituents"` domain.

```
R> dow_constituents <- download_data(
+  domain = "constituents",
+  index = "Dow Jones Industrial Average"
+)
R> download_data(
+  domain = "stock_prices",
+  symbols = dow_constituents$symbol,
+  start_date = "2020-01-01",
+  end_date = "2024-12-31"
+)

# A tibble: 37,710 x 8
```

```
   symbol date        volume  open   low  high close adjusted_close
   <chr>  <date>       <dbl> <dbl> <dbl> <dbl> <dbl>          <dbl>
 1 GS     2020-01-02 3736300  231   230.  235.  234.           201.
 2 GS     2020-01-03 2274500  232.  230.  233.  232.           199.
 3 GS     2020-01-06 3329300  230.  229.  234.  234.           201.
 4 GS     2020-01-07 5255200  235   235.  238.  235.           202.
 5 GS     2020-01-08 3564700  236.  235.  240.  238.           204.
 6 GS     2020-01-09 3980700  241.  240.  243.  243.           208.
 7 GS     2020-01-10 2248100  243.  241.  243.  242.           208.
 8 GS     2020-01-13 3359200  244.  243   246.  245.           211.
 9 GS     2020-01-14 4302800  245.  245.  249.  246.           211.
10 GS     2020-01-15 5411200  242   239.  250.  245.           211.
# i 37,700 more rows
```

Similarly, we can fetch the Fama-French three-factor data from Ken French's Data Library with a single call to download and parse the published archives:

```
R> ff_factors <- download_data(
+  domain = "factors_ff",
+  dataset = "Fama/French 3 Factors",
+  start_date = "1972-01-01",
+  end_date = "2024-12-31"
+)
```

The `start_date` and `end_date` arguments generalize across every dataset type in Table 2 that supports a time dimension.

Public data still requires nontrivial cleaning. The Goyal-Welch (Goyal and Welch 2008) macroeconomic predictors are a typical case: `download_data(domain = "macro_predictors", dataset = "monthly")` fetches the raw spreadsheet at the requested frequency (set `dataset` to `monthly`, `quarterly`, or `annual`), parses dates from the native `yyyymm`, `yyyyq`, or `yyyy` format, strips comma thousand separators, and coerces character columns to numeric. It then constructs the standard predictors used in the equity-premium prediction literature: the log dividend-price ratio (`dp`), dividend yield (`dy`), earnings-price ratio (`ep`), dividend-payout ratio (`de`), the forward equity risk premium (`rp_div`), the term spread (`tms`), and the default-yield spread (`dfy`). The output is the cleaned panel filtered to the requested date range.

For empirical asset pricing work, WRDS is the common source of return and accounting panels. WRDS data are proprietary and require an institutional subscription and credentials. **tidyfinance** handles this setup once. `set_wrds_credentials()` prompts for the username and password and stores them in a project-specific or global `.Renviron` file, and prompts the user to add `.Renviron` to `.gitignore` to prevent credential leakage. Subsequent `download_data()` calls open the database connection automatically. Depending on the user's WRDS account settings, `download_data()` may also trigger a two-factor authentication (2FA) challenge that WRDS sends to the registered device; once approved, the connection proceeds normally.

`download_data()` applies standard cleaning conventions following Bali, Engle, and Murray (2016) and Scheuch *et al.* (2023) automatically. For CRSP monthly data, the function restricts the universe to common equity securities listed on NYSE, AMEX (now NYSE American), or NASDAQ. It computes market capitalization as shares outstanding times price, constructs a

one-month market capitalization lag for portfolio weights, and maps listing exchange codes and Standard Industrial Classification (SIC) codes to readable labels. Excess returns are raw returns minus a risk-free rate that **tidyfinance** constructs from FRED by splicing the 3-month Treasury bill secondary-market rate (pre-2001) with the 4-week Treasury bill secondary-market rate (from 2001 onward); the spliced series is hosted on Hugging Face and refreshed monthly by a scheduled GitHub Actions workflow. The function drops observations with missing returns or market capitalization, and incorporates delisting returns directly, eliminating the need for a separate adjustment. The function defaults to the CRSP 2.0 (CIZ) layout first introduced in July 2022. Alternatively, passing `version = "v1"` returns the legacy (SIZ) layout, which ends in December 2024 (see Schwarz, Walter, and Weiss 2026, for details).

For Compustat annual data, `download_data()` restricts to industrial-format consolidated statements and retains only the most recent observation per firm-year. It constructs book equity in priority order: stockholders' equity when available, else common equity plus preferred stock, else total assets minus total liabilities. The function adds back deferred taxes and investment tax credits, and subtracts the book value of preferred stock.

For TRACE corporate bond transactions, `download_data()` applies the Dick-Nielsen cleaning protocol (Dick-Nielsen 2009) to remove cancellations, corrections, and reversals before returning the trade panel.

Across `download_data()`, every returned panel exposes a single `date` column of `<date>` type, regardless of source. Joining observations from different providers therefore reduces to an equality join on `date` without manual format conversion. The convention is the same across frequencies: daily observations return the trading date, monthly observations the first calendar day of the reference month (regardless of whether it was a trading day), and quarterly and annual observations the first day of the fiscal period-end month as reported by the original source. For Compustat, the latter is the firm's fiscal period-end (`datadate`). Macroeconomic series from FRED keep their original release dates. Where the underlying source uses an internal date that differs from this convention, **tidyfinance** preserves it as an auxiliary column: CRSP 2.0 monthly returns, for example, retain the trading-date `mthcaldt` as `calculation_date` alongside the harmonized `date`. Cross-frequency joins, such as merging monthly returns with annual fundamentals, require the lagging logic described in Section 3.4.

3.4. Variable construction

Most predictive variables in asset pricing studies use lagged information so that, for example, portfolio formation in month t always uses only data observable at the end of month $t-1$. `add_lagged_columns()` automates the required lagging procedure. For each row in the data, it looks back over a configurable window and attaches the most recent observed value of the chosen columns. Figure 1 illustrates the two modes the function supports. By default (`ff_adjustment = FALSE`), shown in Panel A, the window is bounded by `lag` (the minimum delay between observation and use) and `max_lag` (the maximum delay before a stale value is treated as missing). Within this window (the dashed rectangle), the function picks the most recent non-missing value; if no observation falls in the window, the lagged column takes `NA`.

The second mode, enabled by `ff_adjustment = TRUE` and shown in Panel B of Figure 1, implements the Fama-French convention (Fama and French 1993). The convention assumes

accounting variables for fiscal year $t-1$ become public information by June of year t, then uses them for portfolio formation from July of t through June of $t+1$. This timing produces a step-function lag that updates once per year at the June/July boundary, regardless of the underlying fiscal year-end: all months from July through June share the value locked in at the previous rebalancing.

`join_lagged_values()` is a related helper that merges lagged values from one data frame into another using the same windowing logic. It is useful when the target panel and the source panel are at different frequencies, for example, when joining monthly accounting data into a quarterly target panel. The function takes the same `ff_adjustment` argument as `add_lagged_columns()`, so the Fama-French convention can be applied at the join step rather than as a separate transformation.

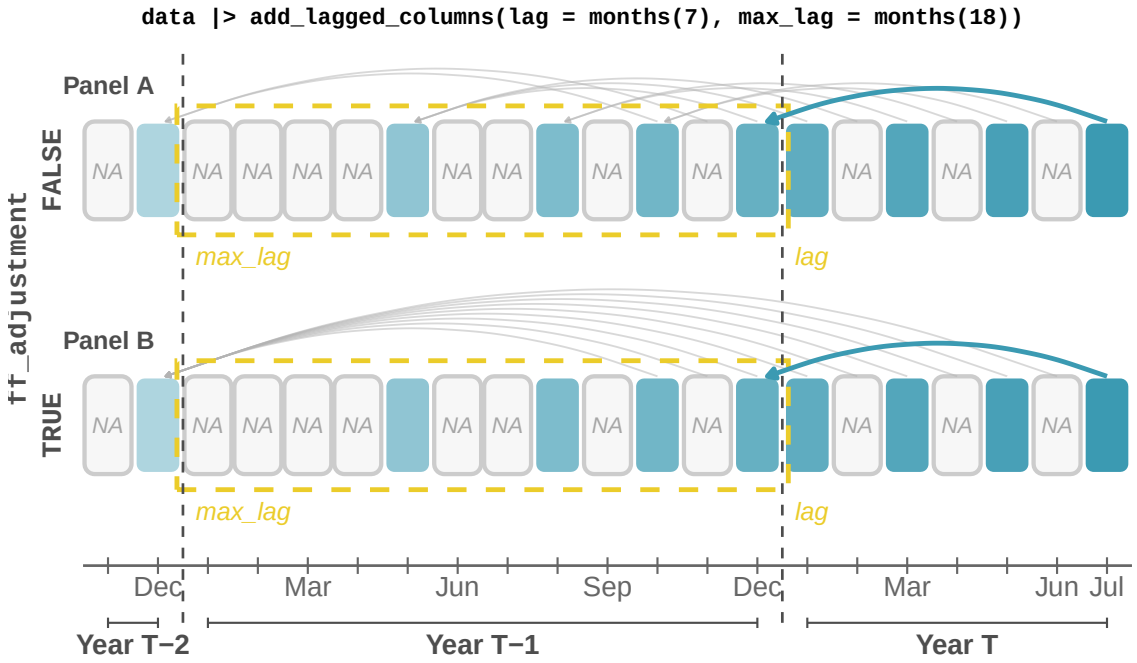


Figure 1: Two lagging mechanisms in `add_lagged_columns()`. Each row represents a data series with observed (filled) and missing (hollow) cells. The dashed rectangle marks the search window from the perspective of the most recent observation (the top row); arrows show which value gets matched to each target month. Panel A (`ff_adjustment = FALSE`) returns the most recent observed value within a window bounded by `lag` and `max_lag`. Panel B (`ff_adjustment = TRUE`) implements the Fama-French convention, where the lagged value updates only at the June/July boundary and is held constant for the next twelve months.

A handful of additional helpers cover routine operations that recur across empirical studies. `winsorize()` and `trim()` handle outliers in cross-sectional variables: the former symmetrically replaces values below the q-th and above the $(1-q)$-th percentile with the corresponding quantile values, while the latter sets them to `NA`. `estimate_model()` fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame, which composes

naturally with `group_by()` for grouped or rolling regressions such as estimating time-series factor exposures stock by stock. `compute_rolling_value()` applies an arbitrary aggregation function over a rolling window defined in calendar time rather than row counts, accommodating irregular observation frequencies and missing periods. `filter_sorting_data()` screens a panel before sorting, dropping observations that fail the universe filters bundled in a `filter_options()` object, such as financial or utility firms or stocks below a minimum price or size; by default it reports how many observations each filter removes, which `quiet = TRUE` suppresses. `implement_portfolio_sort()` calls it internally, but it is also exposed for use as a standalone screening step.

3.5. Portfolio sorting

A portfolio sort groups stocks each period into ordered portfolios according to the level of a sorting variable. The boundaries between portfolios are called *breakpoints*: numeric cutoffs of the sorting variable that determine which portfolio a stock joins. A quintile sort on size, for instance, has four breakpoints at the 20th, 40th, 60th, and 80th percentiles of the cross-sectional market-capitalization distribution. Common sorting variables include size, beta, book-to-market, and momentum. The long-short return between the extreme portfolios is interpreted as a factor-mimicking strategy and serves as the basis for tests of whether the sorting variable forecasts future returns. Portfolio sorts are among the most widely used tools in empirical asset pricing (Bali *et al.* 2016). We expose the procedure through four composable building blocks plus a one-call wrapper.

`compute_breakpoints()` takes a cross-section and a sorting variable and returns the numeric cutoffs that define the portfolio boundaries. The user supplies either a number of portfolios (for example, five for quintiles) or an explicit vector of percentiles, but not both. A `breakpoints_exchanges` argument restricts the universe used to compute breakpoints. The standard convention is NYSE-listed stocks only, which prevents the large cross-section of small NASDAQ firms from dominating the breakpoints (Jensen *et al.* 2023). A related `breakpoints_min_size_threshold` argument excludes stocks below a market-cap quantile from the breakpoint computation, which size-related sorts often require because microcaps would otherwise distort the cutoffs. A `smooth_bunching` flag handles cases where the sorting variable takes the same value for many observations at the bottom or top of the distribution, so that the lowest (or highest) quantile breakpoints coincide. Without intervention, all tied observations would land in a single boundary portfolio while neighboring portfolios remain empty. When `smooth_bunching = TRUE`, the function assigns the bunched observations to the boundary portfolio and recomputes the remaining breakpoints on the subset of observations strictly inside the boundary value, restoring well-defined cutoffs for the interior portfolios.

`assign_portfolio()` maps each observation to a portfolio integer and uses breakpoints supplied either directly or via a breakpoint-computing function (defaulting to `compute_breakpoints()`). In practice, `assign_portfolio()` is called separately for each rebalancing date, so that portfolios are reformed each period rather than fixed once for the whole sample.

`compute_portfolio_returns()` ties the lower-level pieces together. Given a panel of stock data, one or two sorting variables, a sorting method, and `breakpoint_options`, it handles rebalancing, breakpoint computation, portfolio assignment, and return aggregation. It then

returns a complete panel of portfolio returns. For each portfolio-date the function reports three return measures: equal-weighted, value-weighted using `mktcap_lag` as the weight, and capped value-weighted (Jensen *et al.* 2023). The capped version pulls weights above the `cap_weight` percentile (default `0.8` corresponds to 80%) back to that cap, which prevents a few megacap firms from dominating the portfolio. A `min_portfolio_size` argument sets the minimum number of constituents below which the corresponding return is set to `NA`. An optional `rebalancing_month` argument switches from period-by-period rebalancing to annual rebalancing in the specified month. Portfolios formed in that month are then held constant for the following twelve months.

The function supports three sorting schemes, illustrated in Figure 2. A *univariate* sort uses a single variable. An *independent bivariate* sort computes breakpoints on each variable separately and intersects the resulting bins to produce a two-way grid. A *dependent bivariate* sort first partitions the cross-section by the secondary (control) variable and then re-sorts within each bucket on the main variable, which is the appropriate choice when the second variable is treated as a control rather than as an independent dimension of variation. For bivariate sorts, returns are averaged over the secondary variable so that only the main-variable portfolios are returned.

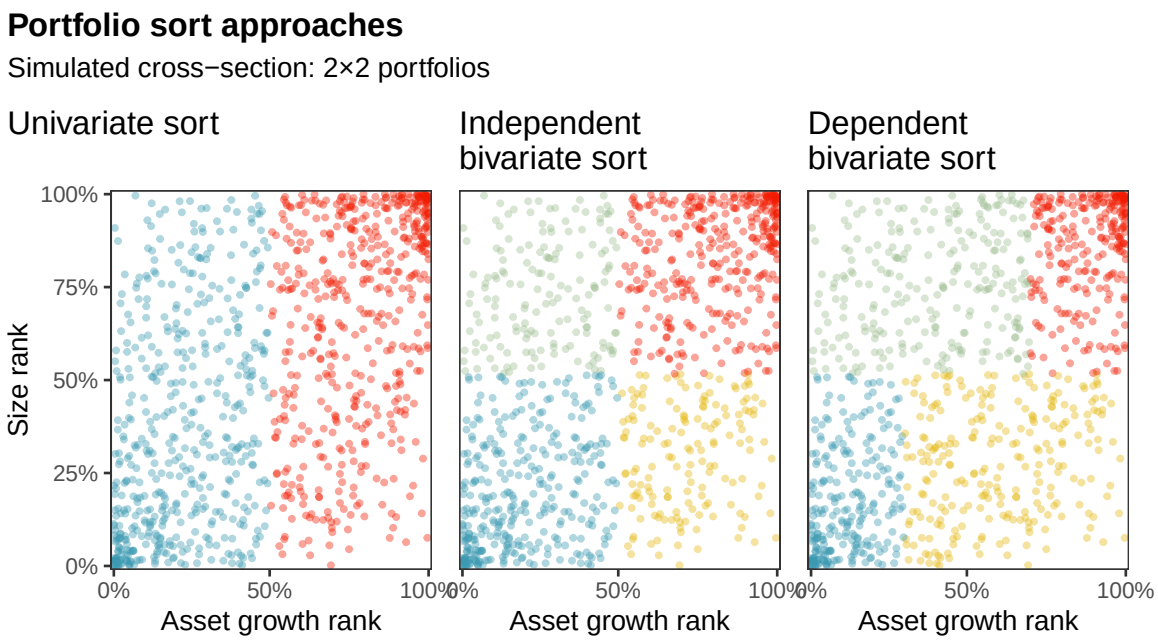


Figure 2: Portfolio sort approaches illustrated on a simulated cross-section. Each panel plots simulated observations assigned to 2 × 2 portfolios based on asset growth rank (x-axis) and size rank (y-axis), both expressed as percentiles. Colors indicate portfolio cell assignment. The univariate sort (left) assigns portfolios on asset growth alone; the independent bivariate sort (center) assigns each dimension independently; the dependent bivariate sort (right) sorts on size first, then on asset growth within size groups.

`compute_long_short_returns()` takes the output of `compute_portfolio_returns()` and computes the spread between the top and bottom portfolios. A `direction` argument controls

the sign and defaults to `"top_minus_bottom"`. The function returns one long-short series per return measure available in the input, so the equal-weighted, value-weighted, and capped value-weighted spreads are produced together. This long-short return is the factor-mimicking spread that the asset-pricing literature typically takes as the object of interest.

For users who want the full pipeline in a single call, `implement_portfolio_sort()` wraps the four functions above. It takes a panel of stock data, one or two sorting variables, a sorting method, and a configuration object, and returns the long-short factor return series alongside the underlying portfolio returns. The lower-level functions remain available for custom workflows that need finer control over breakpoints, weighting, or rebalancing.

4. A case study in factor construction

We walk through the construction of the asset growth factor from raw CRSP and Compustat data to long-short portfolio returns, applying each stage of Table 1 in turn.

Load **tidyfinance** together with a minimal set of supporting packages: **dplyr** (Wickham, François, Henry, Müller, and Vaughan 2026) for data manipulation, **tidyr** (Wickham, Vaughan, and Girlich 2024) for reshaping between long and wide formats, and **lubridate** (Grolemund and Wickham 2011) for date arithmetic.

```
R> library("tidyfinance")
R> library("dplyr")
R> library("tidyr")
R> library("lubridate")
```

4.1. Data download

Constructing the asset growth factor requires two WRDS panels: annual accounting data from Compustat to compute the asset growth signal and monthly stock returns from CRSP to translate the signal into portfolio returns.

CRSP and Compustat are proprietary and require a WRDS subscription. All numbers, tables, and figures in this paper are computed from the real WRDS data with `data_domain <- "wrds"`. Readers without a WRDS subscription can render the entire manuscript against synthetic data of identical schema by setting `data_domain <- "pseudo"` in the chunk below. The pseudo path covers the three WRDS-sourced datasets the case study consumes: monthly CRSP returns, annual Compustat fundamentals, and the CRSP-Compustat linking table. Every `download_data()` call for one of these datasets then routes to an in-package synthetic generator rather than to WRDS, so the document compiles end-to-end without a subscription, credentials, or network access to a paid provider.

The generator preserves column names, dtypes, missingness patterns, panel structure, and approximate scales, so the cleaning, lagging, and sorting code paths exercise the same branches they would on real data, and every figure and table in this paper renders without error. What the generator does not preserve are economic relations. Returns are independent draws around a calibrated mean, and characteristics carry no predictive content for subsequent returns; the

long-short premia recovered against pseudo data are therefore noise. The workflow reproduces; the quantitative results do not.

Two arguments to `download_data()` control the synthetic draws: `seed` fixes the random number generator so that two readers running the same chunk obtain bit-identical pseudo panels, and `n_assets` sets the size of the synthetic cross-section (default `1000L`), trading off compile speed against the realism of cross-sectional statistics. The practical implication is that every chunk in this paper can be inspected, edited, and re-rendered on a laptop with no proprietary data and no credentials; only the headline factor premia require WRDS to reproduce numerically.

Before the first WRDS download, call `set_wrds_credentials()` to store WRDS credentials securely in `.Renviron`; subsequent `download_data()` calls establish the database connection automatically.

```
R> data_domain <- "wrds"
```

```
R> date_start <- as.Date("1972-01-01")
R> date_end <- as.Date("2024-12-31")
```

We restrict the sample to the period from 1972 onward because Compustat coverage is often incomplete before that date, and we end the sample at 2024, the most recent full calendar year at the time of writing.

First, we download monthly stock returns from CRSP. Setting `add_ccm_links = TRUE` appends CRSP-Compustat linking identifiers needed to merge with financial statement data in the next step.

```
R> crsp_monthly <- download_data(
+  domain = data_domain,
+  dataset = "crsp_monthly",
+  version = "v2",
+  start_date = date_start,
+  end_date = date_end,
+  add_ccm_links = TRUE
+)
```

The dataset contains 3,176,966 stock-month observations for 25,347 unique stocks, covering the period from January 1st, 1972 to December 1st, 2024.

Next, we download annual financial statements from Compustat.

```
R> compustat_annual <- download_data(
+  domain = data_domain,
+  dataset = "compustat_annual",
+  start_date = date_start,
+  end_date = date_end,
+  additional_columns = c("at", "ib"),
+  only_usd = TRUE
+) |>
+  select(gvkey, date, at, ib)
```

Annual Compustat data contain 472,061 firm-year observations for 39,329 unique firms, covering the period from January 1st, 1972 to December 31st, 2024. The `additional_columns` argument retrieves total assets (`at`) and income before extraordinary items (`ib`), needed for computing asset growth and applying an earnings filter in later steps. Setting `only_usd = TRUE` restricts the dataset to USD-denominated stocks, ensuring consistency with CRSP.

4.2. Data cleaning and variable construction

We illustrate data cleaning and variable construction using asset growth (AG) as our running example. Asset growth is the year-over-year percentage change in total assets (Cooper *et al.* 2008),

$$AG_t = (AT_t - AT_{t-1})/AT_{t-1},$$

where AT_t is total assets in year t (Compustat field `at`).

The `add_lagged_columns()` function from **tidyfinance** handles temporal alignment by creating a lagged version of total assets grouped by firm identifier (`gvkey`) with a one-year lag. Non-finite values (arising when lagged total assets are zero or missing) become `NA`.

```
R> sorting_variable_data <- compustat_annual |>
+  add_lagged_columns(
+    cols = "at",
+    lag = years(1),
+    by = "gvkey"
+  ) |>
+  mutate(
+    asset_growth = (at - at_lag) / at_lag,
+    asset_growth = if_else(is.finite(asset_growth), asset_growth, NA_real_)
+  ) |>
+  select(gvkey, date, asset_growth, ib)
```

The call relies on two silent defaults: `max_lag` falls back to `lag` (here one year), so only the single observation exactly one year before each row is eligible; `ff_adjustment = FALSE`, so the look-back is the plain calendar window rather than the Fama-French June/July rebalancing convention introduced in Section 3.4.

`sorting_variable_data` contains one observation per firm-year, the natural unit for cross-sectional analysis of annual accounting variables. Asset growth is heavily right-skewed: a small number of firms with dramatic balance-sheet expansions generate extreme positive values. Table 3 reports the pooled cross-sectional summary alongside the time-series average, minimum, and maximum of the corresponding statistic computed cross-sectionally each year.[1] We obtain both views with `create_summary_statistics()`, a **tidyfinance** helper that returns the standard set of cross-sectional summary statistics for a variable: applied to the pooled (winsorized) sample for the first column and grouped by calendar year for the next three. Table 3 assembles the two computations side by side.

[1]The pooled count of 358,036 firm-years is below the 472,061 downloaded in Section 4.1 because the one-year lag in the asset-growth construction drops each firm's first observation.

Statistic	Pooled	Yearly avg	Yearly min	Yearly max
N	358,036	6,885	3,808	9,467
Mean	0.249	0.261	0.048	0.889
SD	0.932	0.980	0.203	3.830
P10	-0.205	-0.196	-0.397	-0.007
P25	-0.041	-0.039	-0.151	0.060
Median	0.061	0.062	0.001	0.132
P75	0.208	0.210	0.105	0.370
P90	0.609	0.631	0.253	1.341

Table 3: Cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). The 'Pooled' column reports the statistic computed on the entire firm-year sample. The 'Yearly' columns summarize the time series of statistic values computed cross-sectionally year by year, taking the average, minimum, and maximum across calendar years. N is the count of firm-year observations (pooled) or per-year observations (yearly columns). Sample: 1973–2024.

```
R> pooled_stats <- sorting_variable_data |>
+  drop_na(asset_growth) |>
+  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
+  create_summary_statistics(asset_growth, detail = TRUE)
R> yearly_stats <- sorting_variable_data |>
+  drop_na(asset_growth) |>
+  group_by(year = year(date)) |>
+  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
+  ungroup() |>
+  create_summary_statistics(asset_growth, by = year, detail = TRUE)
```

The next step merges annual sorting variables onto the monthly CRSP stock panel using `join_lagged_values()`. The function aligns fiscal-year-end data with monthly observations through a configurable lag window. Setting `ff_adjustment = TRUE` implements the Fama-French convention (Fama and French 1993), aligning fiscal-year data from year t with portfolio formation in July of year $t + 1$ through a 12-to-18-month look-back window.

```
R> sorting_data <- crsp_monthly |>
+  join_lagged_values(
+    new_data = sorting_variable_data,
+    id_keys = "gvkey",
+    min_lag = months(7),
+    max_lag = months(18),
+    ff_adjustment = TRUE
+  ) |>
+  select(
+    date,
+    permno,
+    exchange,
```

```
+    siccd,
+    ret_excess,
+    mktcap_lag,
+    asset_growth,
+    ib
+  ) |>
+  filter(date > date_start + months(12) + months(18))
```

The filter removes the initial year used to compute the growth rate plus an additional 18 months to avoid mechanically introduced gaps from the lagging step, leaving a panel of stock-month observations with excess returns, exchange membership, lagged market capitalization, and the lagged sorting variable. The auxiliary columns `siccd` and `ib` support the sample-construction filters in Section 4.3.2.

Figure 3 shows the 33rd, 50th, and 66th percentiles of asset growth over time in the resulting panel; pronounced spikes during economic expansions and contractions highlight the cyclical nature of corporate investment.

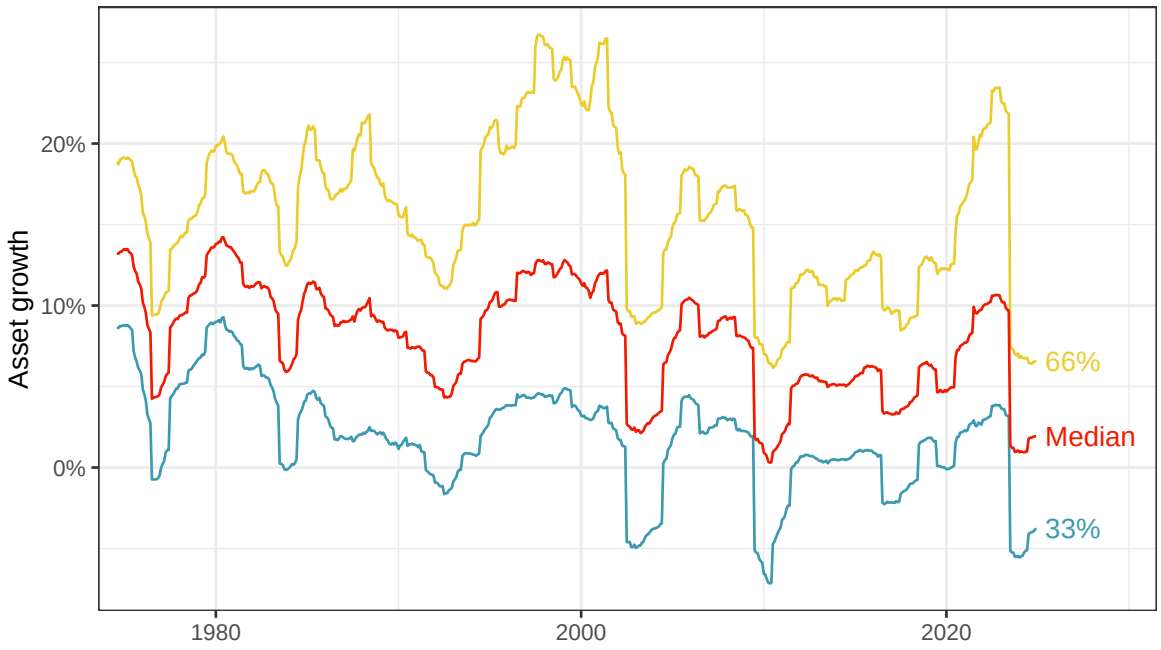


Figure 3: Selected percentiles (33rd, 50th, 66th) of cross-sectional asset growth over time. The y-axis reports annual growth in total assets in percent. Sample: 1974–2024.

With the sorting variable constructed and merged onto the monthly stock panel, we now turn to portfolio sorts, the mechanism that translates firm-level characteristics into tradable factor returns.

4.3. Portfolio sorts

In portfolio sorts, stocks are ranked each period by their sorting variable and divided into quan-

tile portfolios. Portfolio returns are equal-weighted or value-weighted averages of constituent stock returns. The long-short factor return is the difference between the extreme portfolios. For asset growth, the relevant spread is bottom minus top, reflecting the negative premium documented by Cooper *et al.* (2008). The construction involves several methodological choices: which stocks enter the universe, how many portfolios to form, which exchange determines the breakpoints, the weighting scheme, and the rebalancing frequency.

`implement_portfolio_sort()` exposes all of these choices through a single function call. The function takes the panel data, one or two sorting variables, a sorting method, and a `portfolio_sort_options` object that bundles two configuration layers: `filter_options()` controls which observations enter the breakpoint and return computation (for example, excluding financial firms or stocks below a minimum price), and `breakpoint_options()` controls how portfolios are formed (number of portfolios, exchange universe, bunching). Internally, the function applies the sample filters and then calls `compute_portfolio_returns()`, which produces the time series of portfolio returns. A subsequent call to `compute_long_short_returns()` constructs the long-short portfolio.

`compute_portfolio_returns()` itself rests on two lower-level helpers, `compute_breakpoints()` for quantile cutoffs and `assign_portfolio()` for mapping each stock-month to a portfolio number. Both are exported, so researchers who need finer control can call them directly rather than through the wrapper. The two examples below use `implement_portfolio_sort()` throughout.

Baseline specification (univariate sorts)

The simplest call to `implement_portfolio_sort()` requires only a `portfolio_sort_options` object that specifies how to form the breakpoints. In a univariate sort, stocks are ranked on a single characteristic. We illustrate using asset growth as the sorting variable, forming quintile portfolios with breakpoints computed from NYSE-listed stocks only, and rebalancing annually in July.

```
R> options_baseline <- portfolio_sort_options(
+  breakpoint_options_main = breakpoint_options(
+    n_portfolios = 5,
+    breakpoints_exchanges = "NYSE"
+  )
+)
```

The `breakpoint_options()` call specifies that breakpoints come only from NYSE-listed stocks, a common convention that avoids the influence of small AMEX and NASDAQ firms on portfolio cutoffs. We then pass these options to `implement_portfolio_sort()`.

```
R> portfolio_returns <- implement_portfolio_sort(
+  data = sorting_data,
+  sorting_variables = "asset_growth",
+  sorting_method = "univariate",
+  rebalancing_month = 7,
+  portfolio_sort_options = options_baseline
+)
```

Apart from setting `rebalancing_month = 7`, every other choice point falls back to the package default: a cap-weight cutoff at the 80th percentile of market capitalization for the capped value-weighted return, no minimum portfolio size, and no smooth bunching. The standard CRSP/Compustat column names (`permno`, `date`, `ret_excess`, `mktcap_lag`, `exchange`) are picked up automatically through the default `data_options()`. Setting `rebalancing_month = 7` reforms the portfolios each July and holds them for the following twelve months, in line with the standard Fama-French rebalancing convention. This schedule pairs with `ff_adjustment = TRUE` in `join_lagged_values()`: the signal is locked in at the June/July boundary, so July is the first month in which a fresh fiscal-year update is available.

`compute_portfolio_returns()`, called internally, returns a long tibble with one row per portfolio-month and the three weighting columns introduced in Section 3.5: `ret_excess_vw` (value-weighted, VW), `ret_excess_ew` (equal-weighted, EW), and `ret_excess_vw_capped` (capped value-weighted, VWC). Piping the result through `compute_long_short_returns()` gives the long-short portfolio. The `direction = "bottom_minus_top"` argument means the factor is long low-AG stocks and short high-AG stocks, in line with the negative premium documented by Cooper *et al.* (2008).

```
R> factor_returns_ag <- portfolio_returns |>
+  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 4 summarizes the time series of each quintile across all three weighting schemes, with the long-short portfolio appended as the final row.

```
R> return_summary <- function(data) {
+  data |>
+    drop_na() |>
+    summarize(
+      across(
+        c(ret_excess_vw, ret_excess_ew, ret_excess_vw_capped),
+        list(
+          mean = \(x) mean(x) * 12,
+          tstat = \(x) mean(x) / (sd(x) / sqrt(n()))
+        ),
+        .names = "{.col}_{.fn}"
+      ),
+      .groups = "drop"
+    )
+}
R> univariate_sort_table <- bind_rows(
+  portfolio_returns |>
+    group_by(portfolio) |>
+    return_summary() |>
+    mutate(portfolio = as.character(portfolio)),
+  factor_returns_ag |>
+    return_summary() |>
+    mutate(portfolio = "L/S")
+)
```

	Value-weighted		Equal-weighted		Capped value-weighted	
Portfolio	Mean	t statistic	Mean	t statistic	Mean	t statistic
1	0.106	4.448	0.157	4.567	0.112	3.960
2	0.091	4.522	0.128	5.130	0.107	4.572
3	0.085	4.058	0.121	4.999	0.106	4.636
4	0.084	3.807	0.112	4.370	0.100	4.090
5	0.075	2.695	0.054	1.636	0.065	2.098
L/S	0.031	2.080	0.102	8.257	0.047	4.011

Table 4: Time-series averages of annualized monthly excess returns for asset growth quintile portfolios. Portfolios are formed annually in July using NYSE breakpoints. Means are annualized (multiplied by 12). The t statistic tests whether the mean monthly excess return differs from zero. Capped value-weighted pulls weights above the 80th percentile back to that cap. The L/S row is the long-short portfolio (bottom minus top quintile). Sample: 1975–2024.

The value-weighted long-short portfolio earns 3.1% per year (t statistic of 2.08), consistent with the negative premium documented by Cooper *et al.* (2008). The equal-weighted spread is larger, as expected for characteristics that correlate with size. The monotone decline of value-weighted means across quintiles confirms that **tidyfinance** reproduces the literature result from raw WRDS data with a single function call.

Customized specification (bivariate sorts)

The baseline specification above takes nearly every package default and runs a univariate sort to form long-short portfolios. The customized specification below changes both: it overrides several of those defaults through `portfolio_sort_options` and moves from sorting on one variable to sorting on two.

We adopt a specification that combines, in a single call to `implement_portfolio_sort()`, the universe restrictions and breakpoint conventions used in many empirical studies: exclude financials and utilities, exclude firms with non-positive earnings, ignore the smallest 20% of NYSE-listed firms when computing breakpoints, form quintiles on AG within size buckets, use NYSE-only breakpoints, rebalance annually in July, and require at least ten constituents per portfolio.

Bivariate (double) sorts control for a second variable, typically firm size (`mktcap_lag`), when forming portfolios on the main characteristic. We use the dependent variant introduced in Section 3.5 and illustrated in Figure 2: stocks are first split into size buckets, then sorted into asset growth portfolios within each bucket, with returns averaged across the size dimension.

```
R> options_customized <- portfolio_sort_options(
+  filter_options = filter_options(
+    exclude_financials = TRUE,
+    exclude_utilities = TRUE,
+    exclude_negative_earnings = TRUE
+  ),
```

```
+  breakpoint_options_main = breakpoint_options(
+    n_portfolios = 5,
+    breakpoints_exchanges = "NYSE",
+    breakpoints_min_size_threshold = 0.2
+  ),
+  breakpoint_options_secondary = breakpoint_options(
+    n_portfolios = 2,
+    breakpoints_exchanges = "NYSE"
+  )
+)
R> portfolio_returns_customized <- implement_portfolio_sort(
+  data = sorting_data,
+  sorting_variables = c("asset_growth", "mktcap_lag"),
+  sorting_method = "bivariate-dependent",
+  rebalancing_month = 7,
+  portfolio_sort_options = options_customized,
+  min_portfolio_size = 10,
+  quiet = TRUE
+)
R> factor_returns_customized <- portfolio_returns_customized |>
+  compute_long_short_returns(direction = "bottom_minus_top")
```

A single options object (`options_customized`) bundles every methodological choice that distinguishes this specification from the baseline; nothing in the surrounding pipeline changes. Switching from a univariate to a size-controlled dependent double sort is a one-word change in `sorting_method`, and the secondary breakpoint object is supplied alongside the primary one without any additional work. Setting `quiet = TRUE` suppresses the diagnostic messages from `filter_sorting_data()`, which report how many observations each filter drops; setting `quiet = FALSE` (the default) is useful when iterating to track sample-construction effects.

For comparison, we also include a bivariate-dependent sort that uses the default filters and breakpoint conventions, so the only difference from the baseline is the addition of size control. The long-short returns from that specification are appended to the customized one in Table 5.

```
R> options_bivariate <- portfolio_sort_options(
+  breakpoint_options_main = breakpoint_options(
+    n_portfolios = 5,
+    breakpoints_exchanges = "NYSE"
+  ),
+  breakpoint_options_secondary = breakpoint_options(
+    n_portfolios = 2,
+    breakpoints_exchanges = "NYSE"
+  )
+)
R> portfolio_returns_bivariate <- implement_portfolio_sort(
+  data = sorting_data,
+  sorting_variables = c("asset_growth", "mktcap_lag"),
+  sorting_method = "bivariate-dependent",
```

Specification	VW mean	VW t statistic
Baseline (univariate, no filters)	0.031	2.080
Bivariate only (size-controlled, no filters)	0.034	2.868
Customized (size-controlled, filtered)	0.024	1.730

Table 5: Comparison of long-short asset growth returns across three specifications. The baseline is a univariate sort with NYSE breakpoints and no filters. The bivariate-only specification adds a dependent double-sort on size to the baseline but otherwise keeps the same universe and breakpoints. The customized specification adds the filter set (financials, utilities, non-positive earnings) and a 20% size threshold on the breakpoints. All rows use annual July rebalancing; means are annualized value-weighted excess returns. Sample: 1975–2024.

```
+  rebalancing_month = 7,
+  portfolio_sort_options = options_bivariate
+)
R> factor_returns_bivariate <- portfolio_returns_bivariate |>
+  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5 decomposes the customized premium into two layers. Adding the size-controlled double sort alone (the bivariate-only row) lifts the annualized premium from 3.1% to 3.4%, with the t statistic moving from borderline to significant at conventional levels. Controlling for size reveals that most of the AG signal in the cross-section was masked by size dispersion in the univariate sort. Layering the filter set and the 20% breakpoint threshold on top trims the magnitude back to 2.4%, where the spread remains significant at conventional levels: removing financials, utilities, and loss-making firms strips out some of the firms contributing to the unfiltered spread. Across all three specifications, the spread is positive and reflects the negative AG premium reported in Cooper *et al.* (2008), who finds an annual value-weighted long-short premium of about 12.6% over July 1968 to June 2003.[2]

Researchers who want finer-grained control can call the lower-level portfolio-sorting functions in Table 1 directly. They share signatures and semantics with `implement_portfolio_sort()`, so an analysis prototyped against the wrapper can be unrolled into its components without rewriting the pipeline. This separation matters when custom logic applies at one stage only, for example, computing breakpoints separately within each industry, or from a trailing window of past data rather than the contemporaneous cross-section.

Asset pricing tests

A long-short return is one summary of whether asset growth predicts the cross-section of returns. A complementary view comes from Fama-MacBeth regressions (Fama and MacBeth 1973), which estimate the average risk premium associated with a characteristic after running cross-sectional regressions period by period. **tidyfinance** implements the standard procedure in `estimate_fama_macbeth()`, which takes a panel and a model formula, fits one cross-sectional ordinary least squares (OLS) regression per period, and aggregates the period-level coefficients into average risk premia with Newey-West standard errors with automatic lag selection.

[2] Cooper *et al.* (2008) sorts into deciles.

Variable	Risk premium	SE	t statistic	N (months)
Intercept	0.0169	0.0046	3.6599	605
Asset growth	-0.0054	0.0008	-7.0826	605
Log market cap	-0.0014	0.0004	-3.2509	605

Table 6: Fama-MacBeth regression of monthly excess returns on lagged asset growth and lagged log market capitalization. Coefficients are time-series averages of monthly cross-sectional OLS estimates; standard errors are Newey-West (Newey and West 1987) with automatic lag selection. The intercept and slopes on the right-hand-side variables are reported on the monthly scale. Sample: 1974–2024.

Here, we winsorize the AG signal at the 1st and 99th percentiles to keep extreme observations from dominating the cross-sectional regressions and pair it with log market capitalization as a size control.

```
R> fm_data <- sorting_data |>
+  drop_na(asset_growth, mktcap_lag, ret_excess) |>
+  group_by(date) |>
+  mutate(
+    asset_growth = winsorize(x = asset_growth, cut = 0.01),
+    log_mktcap = log(mktcap_lag)
+  ) |>
+  ungroup()

R> fm_results <- estimate_fama_macbeth(
+  data = fm_data,
+  model = "ret_excess ~ asset_growth + log_mktcap"
+)
```

The negative slope on `asset_growth` in Table 6 lines up with the long-short evidence in Table 4 and Table 5: high-AG firms earn lower subsequent returns, conditional on size. As an external benchmark, Cooper *et al.* (2008, Table III, Panel A, Model 1) reports a Fama-MacBeth slope on annual ASSETG of -0.0922 (t statistic of -6.52). Converted to a monthly basis and sign-flipped, their slope is approximately 0.0077 ($= -1 \cdot (-0.0922/12)$), directly comparable to the magnitude of the AG coefficient in our table.[3] Switching to `vcov = "iid"` or supplying `vcov_options = list(lag = 6, prewhite = FALSE)` would change the standard-error treatment without altering the rest of the call, and adding additional regressors (e.g., a beta estimate from `estimate_betas()`, a profitability measure, an idiosyncratic-volatility control) is a one-line extension to the model formula. The same function therefore covers the standard cross-sectional asset-pricing test that complements the portfolio-sort evidence used throughout the rest of the paper.

5. A transparent risk factor library

[3] Cooper *et al.* (2008) additionally controls for book-to-market and 6- and 36-month past returns, both negatively correlated with AG.

5.1. Specification grid

When every construction choice is configurable, no single combination is canonical. Rather than ship one default series and force users to rebuild the pipeline for every variant, we run **tidyfinance** across many defensible combinations and publish the results as a precomputed factor library. The complete construction pipeline is openly available at github.com/tidy-finance/factor-library, so every step from raw data to published series can be inspected and reproduced. Walter *et al.* (2024) and Gruber, Kastner, Voigt, and Weiss (2025), among others, argue for systematic enumeration over construction choices in preference to any single canonical specification.

The library is constructed by running each combination of the choices in Table 7 on the same underlying CRSP-Compustat universe. Beyond the variation Table 7 makes explicit, two universe filters apply across all specifications. An *earnings filter* drops firms with non-positive income before extraordinary items (`ib`), which mitigates the influence of financial distress on the resulting portfolios. An *age filter* drops firms younger than two years, which ensures the sorting variable is based on stable financial information. Both filters are exposed as parameters of `download_data()`; users who want to switch them off can override the default.

Table 7 lists the preprocessing choices that vary across the library. Italic values mark the default for each choice; non-italic values are the alternatives we run.

Preprocessing choice	Argument	Options	Description
Size exclusion	`min_size_quantile`	None; *0.2*	Fraction of the smallest stocks (by market cap) excluded from the portfolio universe.
Exclusion of financials	`exclude_financials`	*No*; Yes	Drops financial-sector stocks (SIC 6000–6999) from the universe.
Exclusion of utilities	`exclude_utilities`	*No*; Yes	Drops utility-sector stocks (SIC 4900–4999) from the universe.
Exclusion of neg. earnings	`exclude_negative_earnings`	*No*; Yes	Drops firms with negative earnings before sorting.
Sorting variable lag	`sorting_variable_lag`	3m; *6m*; FF	Lag applied to the sorting variable before portfolio assignment (e.g., 6m means a 6-month lag).
Rebalancing	`rebalancing`	*Monthly*; Annual	How frequently portfolios are reformed.
Quantile groups (main)	`n_portfolios_main`	3; 5; *10*	Number of quantile groups (e.g., 10 for decile portfolios).

Sort method	`sorting_method`	*Univariate*; Bivariate (dep.); Bivariate (indep.)	Single sort, or dependent or independent double sort on size.
Quantile groups (secondary)	`n_portfolios_secondary`	*NA*; 2; 5	Number of groups for the secondary (size) sort dimension; NA for univariate sorts, 2 or 5 for bivariate sorts.
Breakpoint exchanges	`breakpoints_exchanges`	*NYSE*; All	Exchange(s) used to compute breakpoints. NYSE-only follows the conventional Fama-French approach.
Breakpoint min size threshold	`breakpoints_min_size_threshold`	*NA*; 0.2	Minimum market-cap quantile applied when computing breakpoints; NA disables the screen.
Weighting scheme	`weighting_scheme`	EW; *VW*	Return weighting within portfolios: equal-weighted (EW) or value-weighted (VW).

Table 7: Preprocessing choices in the portfolio sort grid and corresponding `download_data()` arguments. Italic values indicate the default. For double sorts, the secondary quantiles and exchange choices apply to the size dimension only; for univariate sorts, those rows are not applicable. FF refers to the lag procedure of Fama and French (1993).

The factor library covers 50 sorting variables. The selection follows Jensen *et al.* (2023). Table 8 lists all variables with their long-short direction and primary reference; Section A-1 in the Appendix documents the construction of each variable. Each variable runs through every valid combination of the choices in Table 7, producing 841,536 factor return series in total.[4]

Variable	Name		Main reference
52w	52-Week High	↑	George and Hwang (2004)
aci	Abnormal Corporate Investment	↓	Titman, Wei, and Xie (2004)
ami	Amihud Illiquidity	↑	Amihud (2002)
ag	Asset Growth	↓	Cooper *et al.* (2008)
beta	Beta	↑	Fama and MacBeth (1973)
bm	Book-to-Market	↑	Daniel and Titman (2006)

[4]Crossing every choice in Table 7 across the 50 sorting variables defines 864,000 candidate specifications, once the secondary size dimension is collapsed for univariate sorts, where it does not apply. Two variable-specific restrictions account for the remaining gap. Market Equity is excluded from the two bivariate sorts because size already serves as the secondary sort variable, and Earnings-to-Market is excluded from specifications that retain firms with negative earnings because a negative denominator inverts the characteristic's ranking. The result is the 841,536 valid series reported here.

cfm	Cash Flow to Market	↑	Lakonishok, Shleifer, and Vishny (1994)
cbop	Cash-Based Operating Profitability	↑	Ball, Gerakos, Linnainmaa, and Nikolaev (2016)
dbe	Change in Common Equity	↓	Richardson, Sloan, Soliman, and Tuna (2005)
dcol	Change in Current Operating Liabilities	↓	Richardson *et al.* (2005)
dfnl	Change in Financial Liabilities	↓	Richardson *et al.* (2005)
dnoa	Change in Net Operating Assets	↓	Hirshleifer, Hou, Teoh, and Zhang (2004)
dpia	Change in PPE and Intangibles	↓	Lyandres, Sun, and Zhang (2008)
dwc	Change in Working Capital	↓	Richardson *et al.* (2005)
cdi	Composite Debt Issuance	↓	Lyandres *et al.* (2008)
csi	Composite Share Issuance	↓	Daniel and Titman (2006)
dm	Debt to Market	↑	Bhandari (1988)
dtv	Dollar Trading Volume	↓	Brennan, Chordia, and Subrahmanyam (1998)
em	Earnings to Market	↑	Basu (1983)
ebm	Enterprise Book to Market	↑	Penman, Richardson, and Tuna (2007)
gpa	Gross Profits to Assets	↑	Novy-Marx (2013)
hr	Hiring Rate	↓	Belo, Lin, and Bazdresch (2014)
iskew	Idiosyncratic Skewness	↓	Bali *et al.* (2016)
ivol	Idiosyncratic Volatility	↓	Ang, Hodrick, Xing, and Zhang (2006)
dinv	Inventory Changes	↓	Thomas and Zhang (2002)
ig	Investment Growth	↓	Xing (2008)
rev	Long-Term Reversal	↓	De Bondt and Thaler (1985)
me	Market Equity	↓	Banz (1981)
mdr	Maximum Daily Return	↓	Bali, Cakici, and Whitelaw (2011)
mom	Momentum	↑	Fama and French (1996)
ndf	Net Debt Financing	↓	Baker and Wurgler (2006)
ndm	Net Debt to Market	↓	Penman *et al.* (2007)
nef	Net Equity Financing	↓	Baker and Wurgler (2006)
nxf	Net External Financing	↓	Baker and Wurgler (2006)
noa	Net Operating Assets	↓	Hirshleifer *et al.* (2004)
npy	Net Payout Yield	↑	Boudoukh, Michaely, Richardson, and Roberts (2007)
o	O-Score	↓	Ohlson (1980); Dichev (1998)
oa	Operating Accruals	↓	Sloan (1996)
ocm	Operating Cash Flow to Market	↑	Desai, Rajgopal, and Venkatachalam (2004)
ope	Operating Efficiency	↑	Fama and French (2015)
ol	Operating Leverage	↑	Novy-Marx (2011)
rer	Real-Estate Ratio	↑	Titman *et al.* (2004)
rs	Relative Strength	↑	Jegadeesh and Livnat (2006)

rmom	Residual Momentum	↑	Blitz, Huij, and Martens (2011)
roa	Return on Assets	↑	Balakrishnan, Bartov, and Faurel (2010)
roe	Return on Equity	↑	Hou, Xue, and Zhang (2015)
sm	Sales to Market	↑	Barbee Jr, Mukherji, and Raines (1996)
tur	Share Turnover	↓	Datar, Naik, and Radcliffe (1998)
sue	Standardized Unexpected Earnings	↑	Foster, Olsen, and Shevlin (1984)
tbi	Taxable Income to Book Income	↓	Lev and Nissim (2004)

Table 8: Sorting variables in the factor library. The direction column indicates whether the long-short factor return goes long top-minus-bottom (↑) or bottom-minus-top (↓) of the characteristic distribution.

5.2. Access

Researchers retrieve precomputed series without rebuilding the pipeline. **tidyfinance** hosts the library on Hugging Face and exposes it through `download_data()`. The library is a versioned downstream artifact: the manuscript itself consumes it through the same `download_data()` calls, so reproducing this paper requires only a download rather than a rerun of the construction code. Hugging Face retains a full commit history of each repository in the same way GitHub does, so legacy versions of the factor library remain accessible by referencing earlier snapshots; users can therefore reproduce results computed against an older release of the library even after the current version has been updated. An interactive dashboard at factors.tidy-finance.org provides the same series through a browser interface. The companion Python package **tidyfinance** offers an identical API for users who prefer to access the factor library data from Python.

A default call retrieves a single factor by name and lag:

```
R> download_data(
+  domain = "tidyfinance",
+  dataset = "factor_library",
+  sorting_variable = "ag",
+  sorting_variable_lag = "6m",
+  start_date = date_start,
+  end_date = date_end
+)
```

```
# A data frame: 636 x 17
       id date       ret_type     ret sorting_variable min_size_quantile
    <int> <date>     <chr>      <dbl> <chr>                          <dbl>
 1 297554 1972-01-01 vw       -0.0137 ag                               0.2
 2 297554 1972-02-01 vw       -0.0300 ag                               0.2
 3 297554 1972-03-01 vw        0.0346 ag                               0.2
 4 297554 1972-04-01 vw       -0.0487 ag                               0.2
 5 297554 1972-05-01 vw       -0.0669 ag                               0.2
```

```
 6 297554 1972-06-01 vw       -0.0116 ag                                0.2
 7 297554 1972-07-01 vw       -0.0113 ag                                0.2
 8 297554 1972-08-01 vw        0.0302 ag                                0.2
 9 297554 1972-09-01 vw       -0.0255 ag                                0.2
10 297554 1972-10-01 vw        0.0266 ag                                0.2
# i 626 more rows
# i 11 more variables: exclude_financials <lgl>, exclude_utilities <lgl>,
#   exclude_negative_earnings <lgl>, sorting_variable_lag <chr>,
#   rebalancing <chr>, n_portfolios_main <chr>, sorting_method <chr>,
#   breakpoints_min_size_threshold <dbl>, n_portfolios_secondary <dbl>,
#   breakpoints_exchanges <chr>, weighting_scheme <chr>
```

The defaults passed to `dataset = "factor_library"` reflect one common construction choice but may not suit every research question. Verify that the selected combination matches the intended design before drawing conclusions.

Setting `fill_all = TRUE` overrides the default values for all unspecified preprocessing parameters and returns long-short returns for every combination of size cutoff, exclusion filter, rebalancing frequency, sorting method, number of portfolios, breakpoint exchange, and weighting scheme that matches the supplied filters. This call is the entry point for evaluating the robustness of a result across the full grid of plausible preprocessing steps. The call below returns a long data frame of monthly long-short returns for the asset-growth sort, with one row per specification-date pair and a specification `id` that maps back to the underlying construction choices.

```
R> ag_data <- download_data(
+  domain = "tidyfinance",
+  dataset = "factor_library",
+  sorting_variable = "ag",
+  start_date = date_start,
+  end_date = date_end,
+  fill_all = TRUE
+)
```

The call returns 10,990,080 rows, covering 17,280 combinations of the choices in Table 7 that match the supplied filters. The `id` column encodes the construction choices for each specification; the `date` and `ret` columns give the monthly factor returns for that specification.

For finer control, `download_data()` accepts two additional argument patterns. `domain = "tidyfinance"` and `dataset="factor_library_grid"` returns the full table of factor construction specifications hosted on our Hugging Face repository; `domain = "tidyfinance"`, `dataset = "factor_library"`, `ids = selected_ids` retrieves returns for a specific vector `selected_ids` of precomputed specification IDs. Researchers who want to iterate over a custom subset of specifications rather than filter by parameter value will prefer this route.

The example below pins one canonical specification (univariate monthly rebalancing, ten value-weighted NYSE portfolios, six-month signal lag, 20% size threshold, no industry or earnings exclusions) and then downloads the resulting monthly return series for all 50 sorting variables in the library through a single `download_data()` call.

```
R> full_grid <- download_data("tidyfinance", "factor_library_grid")
R> selected_ids <- full_grid |>
+  filter(
+    sorting_method == "univariate",
+    sorting_variable_lag == "6m",
+    rebalancing == "monthly",
+    n_portfolios_main == 10,
+    weighting_scheme == "VW",
+    breakpoints_exchanges == "NYSE",
+    min_size_quantile == 0.2,
+    is.na(breakpoints_min_size_threshold),
+    !exclude_financials,
+    !exclude_utilities,
+    !exclude_negative_earnings
+  ) |>
+  pull(id)
R> return_data <- download_data(
+  "tidyfinance",
+  "factor_library",
+  ids = selected_ids,
+  start_date = date_start,
+  end_date = date_end
+)
```

5.3. Construction-choice sensitivity

We illustrate the kind of sensitivity analysis the library supports by computing a Fama-French three-factor alpha for each specification of the asset-growth factor in `ag_data`. Because the factor return is the long-short payoff of a self-financing portfolio, it is already in excess form, and the risk-free rate does not enter the regression. The Fama-French three factors are available from the earlier `download_data(domain = "factors_ff", dataset = "Fama/French 3 Factors")` call in Section 3.3: `ff_factors` contains the monthly market excess return (`mkt_excess`), the size factor (`smb`), and the value factor (`hml`). For each specification i, we estimate

$$r_{i,t} = \alpha_i + \beta_i \, \mathrm{MKT}_t^e + s_i \, \mathrm{SMB}_t + h_i \, \mathrm{HML}_t + \varepsilon_{i,t}$$

by OLS, where $r_{i,t}$ is the long-short return of specification i in month t. We report the intercept α_i, annualized by multiplying by 12, as the abnormal return that the three-factor model leaves unexplained. Figure 4 plots the histogram of these alphas across all 17,280 specifications.

```
R> ag_alpha <- ag_data |>
+  left_join(ff_factors, join_by(date)) |>
+  group_by(id) |>
+  reframe(estimate_model(
+    pick(everything()),
+    model = "ret ~ mkt_excess + hml + smb"
```

```
+  )) |>
+  select(id, alpha = intercept) |>
+  mutate(alpha = alpha * 12)
R>
R> # Cooper (2008, Table II, Panel C, All Firms) value-weighted decile FF3 alpha,
R> # sign-flipped to the bottom-minus-top convention.
R> cooper_vw_alpha <- 0.084
```

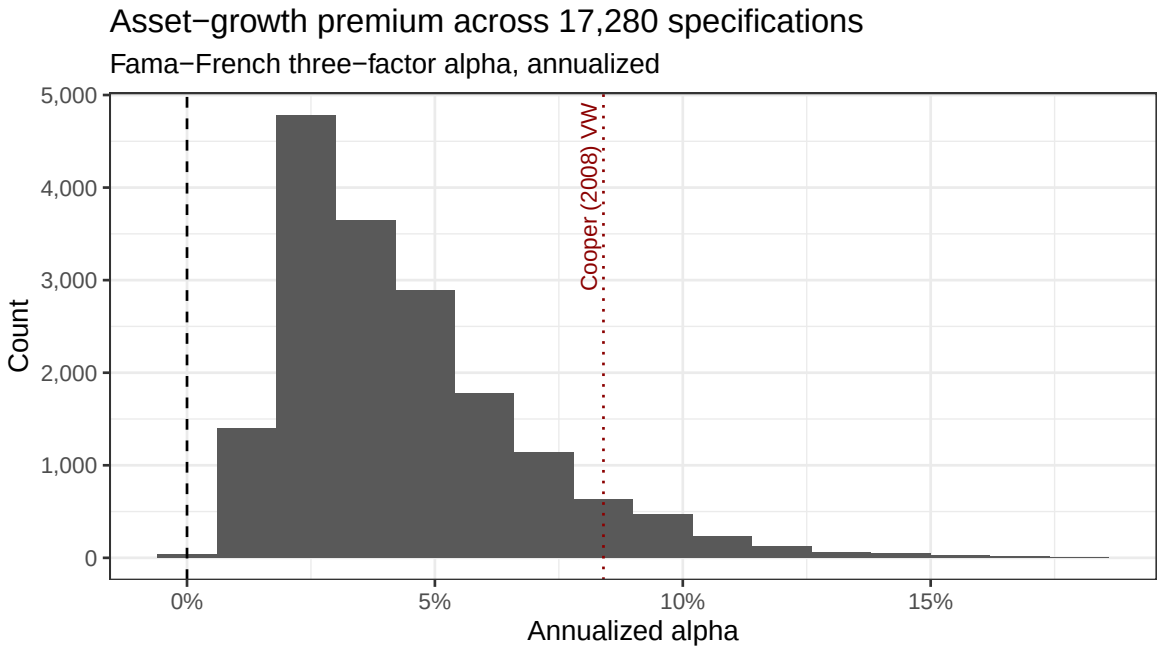


Figure 4: Distribution of asset-growth Fama-French three-factor alphas across construction choices. The histogram shows annualized alphas (monthly intercepts multiplied by 12, expressed in percent on the x-axis) from regressing each of the 17,280 long-short specifications of the asset-growth factor on the market, size, and value factors, estimated by OLS. The dashed vertical line marks zero. The dotted vertical line marks the value-weighted decile FF3 alpha reported in Cooper *et al.* (2008, Table II, Panel C, All Firms), sign-flipped to the bottom-minus-top convention: 8.4% per year over July 1968 to June 2003. Specifications vary as listed in Table 7. Sample: 1972–2024.

The dispersion in Figure 4 shows that construction choices materially shape the estimated premium. Across the 17,280 specifications, annualized alphas average 4.34%, with a median of 3.77%, a cross-specification standard deviation of 2.44%, and a range from -0.31% to 17.67%. Differences of this magnitude across defensible defaults echo the non-standard-errors evidence in Menkveld *et al.* (2024): where the breakpoint is placed, when the portfolio rebalances, and which universe filters apply can move the headline number by more than conventional sampling uncertainty would suggest.

6. Summary and discussion

Empirical asset pricing depends on dozens of construction choices that no consensus pins down. **tidyfinance** makes those choices first-class objects in the workflow: each is an argument, each has a documented default, each can be varied without rebuilding the pipeline. Disagreements over construction become empirical questions, not implicit assumptions buried in code.

We will maintain and extend the package on CRAN. The construction helpers are covered by unit tests that exercise the lagging, breakpoint, and portfolio-aggregation paths on fixture panels, so each release ships against a passing R CMD check. The companion Hugging Face artifacts (the spliced risk-free rate and the precomputed factor library) are refreshed on a scheduled GitHub Actions workflow, so the published series track new WRDS releases without manual intervention.

Companion materials at tidy-finance.org integrate **tidyfinance** into open coursework and self-study tracks for graduate finance and asset pricing in both R and Python. The site also hosts a reviewed channel through which researchers can publish replications and new construction routines, so the catalog of factors and conventions can grow with the literature rather than freezing at the package's release.

Computational details

We obtained the results in this paper using R 4.5.2 with the following packages: **tidyfinance** 0.6.0.9003, **dplyr** 1.2.1 (Wickham *et al.* 2026), **tidyr** 1.3.2 (Wickham *et al.* 2024), **ggplot2** 4.0.3 (Wickham 2016), and **lubridate** 1.9.5 (Grolemund and Wickham 2011). We install development versions of **tidyfinance** with **pak** 0.9.5 (Csárdi and Hester 2024). Figures additionally use **ggforce** 0.5.0 (Pedersen 2024a), **scales** 1.4.0 (Wickham, Pedersen, and Seidel 2023), and **patchwork** 1.3.2 (Pedersen 2024b). CRAN hosts R and all packages at https://CRAN.R-project.org/. **renv** 1.2.3 (Ushey and Wickham 2024) ensures full reproducibility of the computational environment. Reproducing the factor return data requires an institutional subscription to WRDS, as the underlying CRSP and Compustat data are proprietary and cannot be redistributed. A synthetic-data path with identical schema is available via `download_data(domain = "pseudo", ...)` for readers without WRDS access; see Section 4.1.

Reproducing this manuscript takes about 15 minutes: `quarto render jss-manuscript.qmd` pulls the precomputed factor library via `download_data()` and uses the WRDS subset described in Section 4.1, or pseudo data of identical schema for readers without WRDS access. Timings reported here are obtained on Windows 11 x64 (build 26200) with a 8-core processor running R 4.5.2; WRDS extracts are not pre-cached, so the 15-minute figure includes the time to download the underlying CRSP and Compustat panels.

We publish all code, data workflows, and documentation at https://github.com/tidy-finance/jss-multilingual-factor-library. The code is released under the MIT License, which is compatible with the GNU General Public License (GPL).

Acknowledgments

Stefan Voigt is grateful to the Danish Finance Institute for financial support.

References

Akey P, Robertson AZ, Simutin M (2026). "Noisy Factors? The Retroactive Impact of Methodological Changes on the Fama–French Factors." *Review of Finance*. doi:/10.1093/rof/rfag002. Advance access.

Amihud Y (2002). "Illiquidity and Stock Returns: Cross-Section and Time-Series Effects." *Journal of Financial Markets*, **5**(1), 31–56.

Ang A, Hodrick RJ, Xing Y, Zhang X (2006). "The Cross-Section of Volatility and Expected Returns." *The Journal of Finance*, **61**(1), 259–299.

Areal N (2021). ***frenchdata**: Download Data Sets from Kenneth's French Finance Data Library Site*. doi:10.32614/CRAN.package.frenchdata. R package version 0.2.0, URL https://CRAN.R-project.org/package=frenchdata.

Armstrong W, Eddelbuettel D, Laing J, Ooms J (2024). ***Rblpapi**: R Interface to 'Bloomberg'*. doi:10.32614/CRAN.package.Rblpapi.

Baker M, Wurgler J (2006). "Investor Sentiment and the Cross-Section of Stock Returns." *The Journal of Finance*, **61**(4), 1645–1680.

Balakrishnan K, Bartov E, Faurel L (2010). "Post Loss/Profit Announcement Drift." *Journal of Accounting and Economics*, **50**(1), 20–41.

Bali TG, Cakici N, Whitelaw RF (2011). "Maxing Out: Stocks as Lotteries and the Cross-Section of Expected Returns." *Journal of Financial Economics*, **99**(2), 427–446.

Bali TG, Engle RF, Murray S (2016). *Empirical Asset Pricing: The Cross Section of Stock Returns*. John Wiley & Sons.

Ball R, Gerakos J, Linnainmaa JT, Nikolaev V (2016). "Accruals, Cash Flows, and Operating Profitability in the Cross Section of Stock Returns." *Journal of Financial Economics*, **121**(1), 28–45.

Banz RW (1981). "The Relationship Between Return and Market Value of Common Stocks." *Journal of Financial Economics*, **9**(1), 3–18.

Barbee Jr WC, Mukherji S, Raines GA (1996). "Do Sales–Price and Debt–Equity Explain Stock Returns Better Than Book–Market and Firm Size?" *Financial Analysts Journal*, **52**(2), 56–60.

Basu S (1983). "The Relationship Between Earnings' Yield, Market Value and Return for NYSE Common Stocks: Further Evidence." *Journal of Financial Economics*, **12**(1), 129–156.

Belo F, Lin X, Bazdresch S (2014). "Labor Hiring, Investment, and Stock Return Predictability in the Cross Section." *Journal of Political Economy*, **122**(1), 129–177.

Bhandari LC (1988). "Debt/Equity Ratio and Expected Common Stock Returns: Empirical Evidence." *The Journal of Finance*, **43**(2), 507–528.

Blitz D, Huij J, Martens M (2011). "Residual Momentum." *Journal of Empirical Finance*, **18**(3), 506–521.

Boudoukh J, Michaely R, Richardson M, Roberts MR (2007). "On the Importance of Measuring Payout Yield: Implications for Empirical Asset Pricing." *The Journal of Finance*, **62**(2), 877–915.

Boudt K, Cornelissen J, Payseur S, Kleen O, Sjoerup E (2024). ***highfrequency**: Tools for Highfrequency Data Analysis*. doi:10.32614/CRAN.package.highfrequency.

Bradshaw MT, Richardson SA, Sloan RG (2006). "The Relation Between Corporate Financing Activities, Analysts' Forecasts and Stock Returns." *Journal of Accounting and Economics*, **42**(1-2), 53–85.

Brennan MJ, Chordia T, Subrahmanyam A (1998). "Alternative Factor Specifications, Security Characteristics, and the Cross-Section of Expected Stock Returns." *Journal of Financial Economics*, **49**(3), 345–373.

Chen A, Zimmermann T (2022). "Open Source Cross-Sectional Asset Pricing." *Critical Finance Review*, **11**(2), 207–264.

Cooper MJ, Gulen H, Schill MJ (2008). "Asset Growth and the Cross-Section of Stock Returns." *The Journal of Finance*, **63**(4), 1609–1651.

Csárdi G, Hester J (2024). ***pak**: Another Approach to Package Installation*. doi:10.32614/CRAN.package.pak. R package version 0.8.0.

Dancho M, Vaughan D (2026). ***tidyquant**: Tidy Quantitative Financial Analysis*. doi:10.32614/CRAN.package.tidyquant. R package version 1.0.12.

Daniel K, Titman S (2006). "Market Reactions to Tangible and Intangible Information." *The Journal of Finance*, **61**(4), 1605–1643.

Datar VT, Naik NY, Radcliffe R (1998). "Liquidity and Stock Returns: An Alternative Test." *Journal of Financial Markets*, **1**(2), 203–219.

Davis JL, Fama EF, French KR (2000). "Characteristics, Covariances, and Average Returns: 1929 to 1997." *The Journal of Finance*, **55**(1), 389–406.

De Bondt WF, Thaler R (1985). "Does the Stock Market Overreact?" *The Journal of Finance*, **40**(3), 793–805.

Desai H, Rajgopal S, Venkatachalam M (2004). "Value-Glamour and Accruals Mispricing: One Anomaly or Two?" *The Accounting Review*, **79**(2), 355–385.

Dichev ID (1998). "Is the Risk of Bankruptcy a Systematic Risk?" *The Journal of Finance*, **53**(3), 1131–1147.

Dick-Nielsen J (2009). "Liquidity Biases in TRACE." *The Journal of Fixed Income*, **19**(2), 43–55. doi:10.3905/jfi.2009.19.2.043.

Dickerson A, Mueller P, Robotti C (2023). "Priced Risk in Corporate Bonds." *Journal of Financial Economics*, **150**(2), 103707. doi:10.1016/j.jfineco.2023.103707.

Eddelbuettel D, Nguyen K, Leitch T (2024). ***RQuantLib**: R Interface to the 'QuantLib' Library*. doi:10.32614/CRAN.package.RQuantLib.

Fama EF, French KR (1992). "The Cross-Section of Expected Stock Returns." *The Journal of Finance*, **47**(2), 427–465.

Fama EF, French KR (1993). "Common Risk Factors in the Returns on Stocks and Bonds." *Journal of Financial Economics*, **33**(1), 3–56.

Fama EF, French KR (1996). "Multifactor Explanations of Asset Pricing Anomalies." *The Journal of Finance*, **51**(1), 55–84.

Fama EF, French KR (2015). "A Five-Factor Asset Pricing Model." *Journal of Financial Economics*, **116**(1), 1–22.

Fama EF, French KR (2023). "Production of U.S. Rm-Rf, SMB, and HML in the Fama-French Data Library." Chicago Booth Research Paper No. 23-22, Fama-Miller Working Paper. Available at SSRN: https://ssrn.com/abstract=4629613 or DOI: http://dx.doi.org/10.2139/ssrn.4629613.

Fama EF, MacBeth JD (1973). "Risk, Return, and Equilibrium: Empirical Tests." *Journal of Political Economy*, **81**(3), 607–636.

Foster G, Olsen C, Shevlin T (1984). "Earnings Releases, Anomalies, and the Behavior of Security Returns." *The Accounting Review*, **59**(4), 574–603.

Gao X, Ritter JR (2010). "The Marketing of Seasoned Equity Offerings." *Journal of Financial Economics*, **97**(1), 33–52.

George TJ, Hwang CY (2004). "The 52-Week High and Momentum Investing." *The Journal of Finance*, **59**(5), 2145–2176.

Goyal A, Welch I (2008). "A Comprehensive Look at the Empirical Performance of Equity Premium Prediction." *The Review of Financial Studies*, **21**(4), 1455–1508.

Grolemund G, Wickham H (2011). "Dates and Times Made Easy with **lubridate**." *Journal of Statistical Software*, **40**(3), 1–25. doi:10.18637/jss.v040.i03.

Gruber L, Kastner G, Voigt S, Weiss P (2025). "Uncertainty Everywhere: Integrating Conceptual Uncertainty in the Stochastic Discount Factor." Working paper.

Hautsch N, Menkveld AJ, Voigt S (2026). "Market Responses to a VIX Shock." Available at SSRN: https://ssrn.com/abstract=6298780.

Hirshleifer D, Hou K, Teoh SH, Zhang Y (2004). "Do Investors Overvalue Firms with Bloated Balance Sheets?" *Journal of Accounting and Economics*, **38**, 297–331.

Hou K, Mo H, Xue C, Zhang L (2021). "An Augmented Q-Factor Model with Expected Growth." *Review of Finance*, **25**(1), 1–41.

Hou K, Xue C, Zhang L (2015). "Digesting Anomalies: An Investment Approach." *The Review of Financial Studies*, **28**(3), 650–705.

Hou K, Xue C, Zhang L (2020). "Replicating Anomalies." *The Review of Financial Studies*, **33**(5), 2019–2133.

Hribar P, Collins DW (2002). "Errors in Estimating Accruals: Implications for Empirical Research." *Journal of Accounting Research*, **40**(1), 105–134.

Jegadeesh N, Livnat J (2006). "Revenue Surprises and Stock Returns." *Journal of Accounting and Economics*, **41**(1-2), 147–171.

Jensen TI, Kelly B, Pedersen LH (2023). "Is There a Replication Crisis in Finance?" *The Journal of Finance*, **78**(5), 2465–2518.

Lakonishok J, Shleifer A, Vishny RW (1994). "Contrarian Investment, Extrapolation, and Risk." *The Journal of Finance*, **49**(5), 1541–1578.

Lev B, Nissim D (2004). "Taxable Income, Future Earnings, and Equity Values." *The Accounting Review*, **79**(4), 1039–1074.

Lyandres E, Sun L, Zhang L (2008). "The New Issues Puzzle: Testing the Investment-Based Explanation." *The Review of Financial Studies*, **21**(6), 2825–2855.

Menkveld AJ, *et al.* (2024). "Non-Standard Errors." *The Journal of Finance*, **79**(3), 2339–2390.

Newey WK, West KD (1987). "A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix." *Econometrica*, **55**(3), 703–708.

Novy-Marx R (2011). "Operating Leverage." *Review of Finance*, **15**(1), 103–134.

Novy-Marx R (2013). "The Other Side of Value: The Gross Profitability Premium." *Journal of Financial Economics*, **108**(1), 1–28.

Ohlson JA (1980). "Financial Ratios and the Probabilistic Prediction of Bankruptcy." *Journal of Accounting Research*, **18**(1), 109–131.

Pedersen TL (2024a). ***ggforce**: Accelerating **ggplot2***. doi:10.32614/CRAN.package.ggforce. R package version 0.4.2.

Pedersen TL (2024b). ***patchwork**: The Composer of Plots*. doi:10.32614/CRAN.package.patchwork. R package version 1.2.0.

Penman SH, Richardson SA, Tuna I (2007). "The Book-to-Price Effect in Stock Returns: Accounting for Leverage." *Journal of Accounting Research*, **45**(2), 427–467.

Peterson BG, Carl P (2026). ***PerformanceAnalytics**: Econometric Tools for Performance and Risk Analysis*. doi:10.32614/CRAN.package.PerformanceAnalytics. R package version 2.1.0.

Richardson SA, Sloan RG, Soliman MT, Tuna I (2005). "Accrual Reliability, Earnings Persistence and Stock Prices." *Journal of Accounting and Economics*, **39**(3), 437–485.

Ryan JA, Ulrich JM (2025). ***quantmod**: Quantitative Financial Modelling Framework*. doi:10.32614/CRAN.package.quantmod. R package version 0.4.28.

Scheuch C, Voigt S, Weiss P (2023). *Tidy Finance with R*. 1st edition. Chapman and Hall/CRC. doi:10.1201/b23237.

Scheuch C, Voigt S, Weiss P, Frey C (2024). *Tidy Finance with Python*. 1st edition. Chapman and Hall/CRC. doi:10.1201/9781032684307.

Schwarz P, Walter D, Weiss P (2026). "Rewriting CRSP's History: Impact of Altered Monthly Returns on Asset Pricing." *Journal of Financial and Quantitative Analysis*, p. 1–60. doi:10.1017/S0022109026102774.

Sloan RG (1996). "Do Stock Prices Fully Reflect Information in Accruals and Cash Flows about Future Earnings?" *The Accounting Review*, **71**(3), 289–315.

Soebhag A, Van Vliet B, Verwijmeren P (2024). "Non-standard errors in asset pricing: Mind your sorts." *Journal of Empirical Finance*, **78**, 101517.

Thomas JK, Zhang H (2002). "Inventory Changes and Future Returns." *Review of Accounting Studies*, **7**(2), 163–187.

Titman S, Wei KJ, Xie F (2004). "Capital Investments and Stock Returns." *Journal of Financial and Quantitative Analysis*, **39**(4), 677–700.

Tuzel S (2010). "Corporate Real Estate Holdings and the Cross-Section of Stock Returns." *The Review of Financial Studies*, **23**(6), 2268–2302.

Ushey K, Wickham H (2024). ***renv**: Project Environments*. doi:10.32614/CRAN.package.renv. R package version 1.0.7.

Walter D, Weber R, Weiss P (2024). "Methodological Uncertainty in Portfolio Sorts." Available at SSRN: https://ssrn.com/abstract=4164117.

Wickham H (2014). "Tidy Data." *Journal of Statistical Software*, **59**(10), 1–23. doi:10.18637/jss.v059.i10.

Wickham H (2016). ***ggplot2**: Elegant Graphics for Data Analysis*. Springer-Verlag, New York. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org.

Wickham H, François R, Henry L, Müller K, Vaughan D (2026). ***dplyr**: A Grammar of Data Manipulation*. doi:10.32614/CRAN.package.dplyr. R package version 1.2.0.

Wickham H, Pedersen TL, Seidel D (2023). ***scales**: Scale Functions for Visualization*. doi:10.32614/CRAN.package.scales. R package version 1.3.0.

Wickham H, Vaughan D, Girlich M (2024). ***tidyr**: Tidy Messy Data*. R package version 1.3.1, URL https://tidyr.tidyverse.org.

Wuertz D, Setz T, Chalabi Y, Chen W (2023). ***fPortfolio**: Rmetrics — Portfolio Selection and Optimization*. doi:10.32614/CRAN.package.fPortfolio.

Xing Y (2008). "Interpreting the Value Effect Through the Q-Theory: An Empirical Investigation." *The Review of Financial Studies*, **21**(4), 1767–1795.

Appendix

A-1. Variable construction

A-1.1. Financing

CDI - Composite Debt Issuance

Following Lyandres *et al.* (2008), we compute composite debt issuance as the logarithmic growth rate of book debt (current debt, `dlc`, plus long-term debt, `dltt`) over the last five years:

$$CDI_t = \log(DLC_t + DLTT_t) - \log(DLC_{t-60} + DLTT_{t-60}).$$

We replace missing values of `dlc` and `dltt` with zero. The sum of `dlc` and `dltt` must be strictly positive; otherwise, the variable is set to missing.

CSI - Composite Share Issuance

Following Daniel and Titman (2006), we measure composite share issuance from CRSP data as the difference between the change in market equity (ME) (i.e., the shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD) and the cumulative log return (where r is a simple return) in each month from t to $t-60$, i.e.,

$$CSI_t = \log(ME_t/ME_{t-60}) - \sum_{\tau=t-59}^{t} \log(1+r_\tau).$$

DBE - Change in Common Equity

Following Richardson *et al.* (2005), we calculate the change in common equity as the change in common equity (`ceq`) over one year scaled by lagged assets (`at`), i.e.,

$$DBE_t = \frac{CEQ_t - CEQ_{t-12}}{AT_{t-12}}.$$

DCOL - Change in Current Operating Liabilities

Following Richardson *et al.* (2005), we measure the change in current operating liabilities as the change in current operating liabilities (i.e., current liabilities, `lct`, minus current debt, `dlc`) over one year scaled by lagged assets (`at`), i.e.,

$$DCOL_t = \frac{(LCT_t - DLC_t) - (LCT_{t-12} - DLC_{t-12})}{AT_{t-12}}.$$

We set the missing values in `dlc` to zero.

DFNL - Change in Financial Liabilities

Following Richardson *et al.* (2005), we measure the change in financial liabilities (i.e., the sum of current debt, `dlc`, plus long-term debt, `dltt`, plus preferred stocks, `pstk`) over one year scaled by lagged assets (`at`), i.e.,

$$DFNL_t = \frac{(DLC_t + DLTT_t + PSTK_t) - (DLC_{t-12} + DLTT_{t-12} + PSTK_{t-12})}{AT_{t-12}}.$$

We replace the missing values of `dlc`, `dltt`, and `pstk` with zero, if at least one of them is available.

NDF - Net Debt Financing

Following Bradshaw, Richardson, and Sloan (2006), we measure net debt financing as cash proceeds from the issuance of long-term debt (`dltis`) minus cash payments for long-term debt reductions (`dltr`) plus net changes in current debt (`dlcch`) scaled by the average total assets (`at`) over the current and the last year, i.e.,

$$NDF_t = \frac{DLTIS_t - DLTR_t + DLCCH_t}{\frac{1}{2}(AT_t + AT_{t-12})}.$$

We replace the missing values of `dlcch` with zero.

NEF - Net Equity Financing

Following Bradshaw *et al.* (2006), we measure net equity financing as proceeds from the sale of common and preferred stocks (`sstk`) minus payments for the repurchase of common and preferred stocks (`prstkc`) minus cash payments for dividends (`dv`) scaled by the average total assets (`at`) over the current and the last year, i.e.,

$$NEF_t = \frac{SSTK_t - PRSTKC_t - DV_t}{\frac{1}{2}(AT_t + AT_{t-12})}.$$

NXF - Net External Financing

Following Bradshaw *et al.* (2006), we measure net external financing as the sum of net debt financing (NDF) and net equity financing (NEF) as defined above.

A-1.2. Intangibles

HR - Hiring Rate

Following Belo *et al.* (2014), we measure the hiring rate as the change in the number of employees (`emp`) over one year scaled by the average number of employees, i.e.,

$$HR_t = \frac{EMP_t - EMP_{t-12}}{\frac{1}{2}(EMP_t + EMP_{t-12})}.$$

We exclude firms with a hiring rate of zero.

OL - Operating Leverage

Following Novy-Marx (2011), we measure operating leverage as cost of goods sold (`cogs`) plus selling, general, and administrative expenses (`xsga`), both scaled by current total assets (`at`), i.e.,

$$OL_t = \frac{COGS_t + XSGA_t}{AT_t}.$$

RER - Real-Estate Ratio

Following Tuzel (2010), we measure the real-estate ratio in two subperiods differently.

(a) Before 1983, the real-estate ratio is the sum of buildings (`ppenb`) and capital leases (`ppenls`) scaled by net property, plant, and equipment (`ppent`), i.e.,

$$RER_t = \frac{PPENB_t + PPENLS_t}{PPENT_t}.$$

(b) After the end of 1983, the real-estate ratio is the sum of buildings at cost (`fatb`) and leases at cost (`fatl`), both divided by gross property, plant, and equipment (`ppegt`), i.e.,

$$RER_t = \frac{FATB_t + FATL_t}{PPEGT_t}.$$

Subsequently, we winsorize the real-estate ratios in each fiscal year at the first and 99th percentile. The industry-adjusted real-estate ratio is obtained by subtracting the industry average real-estate ratio from each stock's real-estate ratio. We use 2-digit SIC codes to assign stocks to industries. We always require at least five observations to calculate the industry average each year.

A-1.3. Investment

ACI - Abnormal Corporate Investment

Following Titman *et al.* (2004), we measure abnormal corporate investments using the variable CE, which is capital expenditures (`capx`) scaled by sales (`sale`), then ACI is given by CE scaled by CE from the previous three years minus one, i.e.,

$$ACI_t = \frac{CE_t}{\frac{1}{3}(CE_{t-12} + CE_{t-24} + CE_{t-36})} - 1.$$

We follow Hou *et al.* (2020) and exclude stocks with sales below 10 million dollars.

AG - Asset Growth

Following Cooper *et al.* (2008), we measure asset growth as the change in total assets (`at`) over one year scaled by lagged assets (`at`), i.e.,

$$AG_t = \frac{AT_t - AT_{t-12}}{AT_{t-12}}.$$

DINV - Inventory Changes

Following Thomas and Zhang (2002), we measure the change in inventory as the annual change in inventories (`invt`) scaled by average total assets (`at`) over the current and the last year, i.e.,

$$DINV_t = \frac{INVT_t - INVT_{t-12}}{\frac{1}{2}(AT_t + AT_{t-12})}.$$

DNOA - Change in Net Operating Assets

Following Hirshleifer *et al.* (2004), we measure net operating assets (defined as NOA^{int}) as

$$NOA_t^{int} = (AT_t - CHE_t) - (AT_t - DLC_t - DLTT_t - MIB_t - PSTK_t - CEQ_t),$$

where `at` corresponds to total assets, `che` to cash and short-term investments, `dlc` to current debt, `dltt` to long-term debt, `mib` to minority interests, `pstk` to the value of preferred stocks, and `ceq` to common equity. Missing values in `dlc`, `dltt`, `mib`, and `pstk` are set to zero. Then, the change in net operating assets is the difference between net operating assets of fiscal year t and fiscal year $t-12$ scaled by lagged assets (`at`), i.e.,

$$DNOA_t = \frac{NOA_t^{int} - NOA_{t-12}^{int}}{AT_{t-12}}.$$

DPIA - Change in Property, Plant, Equipment, and Inventory

Following Lyandres *et al.* (2008), we measure the change in property, plant, equipment, and inventory by combining the change in gross property, plant, and equipment (`ppegt`) with the annual change in inventory (`invt`) and scaling this sum by lagged total assets (`at`), i.e.,

$$DPIA_t = \frac{(PPEGT_t - PPEGT_{t-12}) + (INVT_t - INVT_{t-12})}{AT_{t-12}}.$$

DWC - Change in Net Non-Cash Working Capital

Following Richardson *et al.* (2005), we measure non-cash working capital as

$$WC_t = (ACT_t - CHE_t) - (LCT_t - DLC_t),$$

where `act` corresponds to current assets, `che` to cash, `lct` to current liabilities, and `dlc` to short-term debt. We set the missing values in `dlc` to zero. Then, the change in net non-cash working capital corresponds to the change in WC from fiscal year $t-12$ to fiscal year t scaled by lagged total assets (`at`), i.e.,

$$DWC_t = \frac{WC_t - WC_{t-12}}{AT_{t-12}}.$$

IG - Investment Growth

Following Xing (2008), we measure investment growth as the annual change in capital expenditures (`capx`) scaled by lagged capital expenditures, i.e.,

$$IG_t = \frac{CAPX_t - CAPX_{t-12}}{CAPX_{t-12}}.$$

NOA - Net Operating Assets

Following Hirshleifer *et al.* (2004), we measure net operating assets (defined as NOA^{int}) as

$$NOA_t^{int} = (AT_t - CHE_t) - (AT_t - DLC_t - DLTT_t - MIB_t - PSTK_t - CEQ_t),$$

where `at` corresponds to total assets, `che` to cash and short-term investments, `dlc` to current debt, `dltt` to long-term debt, `mib` to minority interests, `pstk` to the value of preferred stocks, and `ceq` to common equity. Missing values in `dlc`, `dltt`, `mib`, and `pstk` are set to zero. Then, net operating assets are the net operating assets of fiscal year t scaled by lagged assets (`at`), i.e.,

$$NOA_t = \frac{NOA_t^{int}}{AT_{t-12}}.$$

OA - Operating Accruals

Following Sloan (1996) (before 1988) and Hribar and Collins (2002) (after 1988), we measure operating accruals until 1988 as

$$OA_t = \frac{(\Delta ACT_t - \Delta CHE_t) - (\Delta LCT_t - \Delta DLC_t - \Delta TXP_t) - DP_t}{AT_{t-12}},$$

where `act` is current assets, `che` cash, `lct` current liabilities, `dlc` short-term debt, `txp` taxes payable, and `dp` depreciation and amortization. Moreover, we replace the missing values in `dlc` and `txp` with zero. Δ represents changes over one year. From 1988 onwards, we measure operating accruals as

$$OA_t = \frac{NI_t - OANCF_t}{AT_{t-12}},$$

where `ni` is net income and `oancf` corresponds to net cash flow from operations.

A-1.4. Momentum

MOM - Return Momentum

Following Fama and French (1996), we measure return momentum from monthly CRSP returns (where r is a simple return) as the cumulative return from month $t-11$ to month $t-1$, skipping the most recent month, i.e.,

$$MOM_t = \exp\left(\sum_{\tau=t-11}^{t-1} \log(1+r_\tau)\right) - 1.$$

RMOM - Residual Momentum

Following Blitz *et al.* (2011), we define the 11-month residual momentum (RMOM) in each month and for each stock as cumulative residual returns from month $t-11$ to month $t-1$, scaled by the standard deviation of residual returns over the same time horizon. Residual returns ϵ are obtained in each month from regressing monthly simple, excess stock returns from month $t-1$ to month $t-36$ on the Fama-French three-factor model. Throughout these rolling regressions, we always require 36 monthly returns.

RS - Revenue Surprise

Following Jegadeesh and Livnat (2006), we measure the change in revenues per share (DRPS) from quarterly Compustat data as quarterly sales (`saleq`) scaled by the product of quarterly shares outstanding (`cshprq`) times their cumulative adjustment factor (`ajexq`), i.e.,

$$DRPS_q = \frac{SALEQ_q}{CSHPRQ_q \cdot AJEXQ_q} - \frac{SALEQ_{q-4}}{CSHPRQ_{q-4} \cdot AJEXQ_{q-4}}.$$

Then, the revenue surprise corresponds to the change in revenues per share over the last four quarters scaled by the standard deviation of the change in revenues per share over the last eight quarters. We require at least six quarterly observations for this rolling standard deviation and remove infinite values. Finally, the earnings announcement date has to be after the fiscal quarter end date.[5]

SUE - Standardized Unexpected Earnings

Following Foster *et al.* (1984), we measure unexpected earnings per share (UES) from quarterly Compustat data as the change in split-adjusted earnings (which are equal to quarterly earnings per share, `epspxq`, scaled by the correction factor for shares outstanding, `ajexq`) per share from its value four quarters ago, i.e.,

$$UES_q = \frac{EPSPXQ_q}{AJEXQ_q} - \frac{EPSPXQ_{q-4}}{AJEXQ_{q-4}}.$$

Then, standardized unexpected earnings are defined as unexpected earnings per share divided by the standard deviation of unexpected earnings per share over the previous eight quarters. We require at least six quarterly observations for this rolling standard deviation. Finally, the earnings announcement date has to be after the fiscal quarter end date.[6]

[5]We follow the convention of Hou *et al.* (2020) and do not estimate a random walk model with drift to compute the surprise.

[6]We follow the convention of Hou *et al.* (2020) and do not estimate a random walk model with drift to compute the surprise.

52W - 52-Week High

Following George and Hwang (2004), we measure the 52-week high as month t's closing split-adjusted stock price scaled by the highest daily split-adjusted stock price over the previous twelve months. We only consider stock-month pairs with more than three valid price observations.

REV - Long-Term Reversal

Following De Bondt and Thaler (1985), we measure the long-term reversal effect by the cumulative returns from month $t-60$ to month $t-13$, i.e.,

$$REV_t = \exp\left(\sum_{\tau=t-60}^{t-13} \log(1+r_\tau)\right) - 1.$$

A-1.5. Profitability

CBOP - Cash-Based Operating Profitability

Following Ball *et al.* (2016), we measure cash-based operating profitability as

$$\begin{aligned} CBOP_t =& (REVT_t - COGS_t - XSGA_t + XRD_t \\ & - \Delta RECT_t - \Delta INVT_t - \Delta XPP_t + \Delta DRC_t \\ & + \Delta DRLT_t + \Delta AP_t + \Delta XACC_t)/AT_t, \end{aligned}$$

where `revt` is total revenue, `cogs` are cost of goods sold, `xsga` are selling, general, and administrative expenses, and `xrd` are R&D expenses. Moreover, Δ refers to one-year changes, which we measure for accounts receivable (`rect`), inventory (`invt`), prepaid expenses (`xpp`), deferred revenues (the sum of current, `drc`, and long-term, `drlt`), trade accounts payable (`ap`), and accrued expenses (`xacc`). We set the missing values of `xrd` and all missing changes to zero.

GPA - Gross Profits to Assets

Following Novy-Marx (2013), we measure gross profits to assets as total revenues (`revt`) minus cost of goods sold (`cogs`) scaled by current total assets (`at`), i.e.,

$$GPA_t = \frac{REVT_t - COGS_t}{AT_t}.$$

O - Ohlson's O-score

Following Ohlson (1980), we measure the O-score with the following linear relation:

$$
\begin{aligned}
O_t = &-1.32 - 0.407 \cdot \log(AT_t) + 6.03 \cdot \frac{DLC_t + DLTT_t}{AT_t} \\
&- 1.43 \cdot \frac{ACT_t - LCT_t}{AT_t} + 0.076 \cdot \frac{LCT_t}{AT_t} \\
&- 1.72 \cdot \mathbf{1}_{LT_t > AT_t} - 2.37 \cdot \frac{NI_t}{AT_t} - 1.83 \cdot \frac{PI_t + DP_t}{LT_t} \\
&+ 0.285 \cdot \mathbf{1}_{NI_t<0 \text{ and } NI_{t-12}<0} - 0.521 \cdot \frac{NI_t - NI_{t-12}}{|NI_t| + |NI_{t-12}|},
\end{aligned}
$$

where `at` corresponds to total assets, `dlc` to short-term debt, `dltt` to long-term debt, `act` to current assets, `lct` to current liabilities, `lt` to total liabilities, `pi` to pretax income, `dp` to depreciation and amortization, and `ni` to net income. We winsorize all variables except for dummy variables at the first and 99th percentile of their respective distribution.

OPE - Operating Profits to Book Equity

Following Fama and French (2015), we measure operating profits to book equity as

$$
OPE_t = \frac{REVT_t - COGS_t - XSGA_t - XINT_t}{BE_t},
$$

where `revt` corresponds to total revenues, `cogs` to cost of goods sold, `xsga` to selling, general and administrative expenses, `xint` to interest expenses, and BE to book equity. The definition of book equity is (a) the book equity of shareholders plus (b) deferred taxes and investment tax credit (Compustat item `txditc` or `txdb` + `itcb` if `txditc` is unavailable) minus (c) the book value of preferred stock. Regarding (a), the book equity of shareholders is shareholders' equity (`seq`), or the sum of common equity (`ceq`) and the par value of preferred stock (`pstk`), or, if all previous items are unavailable, total assets (`at`) minus total liabilities (`lt`). Regarding (b), deferred taxes (`txditc`) or investment tax credits (`txdb` plus `itcb`) if deferred taxes are unavailable. Regarding (c), the book value of preferred stock corresponds in the following order either to the redemption value (`pstkrv`), or the liquidation value (`pstkl`), or if all previous items are unavailable to the par value (`pstk`). We replace the missing values in items (b) and (c) with zero. Moreover, missing values in `cogs`, `xsga`, and `xint` are set to zero.

ROA - Return on Assets

Following Balakrishnan *et al.* (2010), we measure return on assets as quarterly income before extraordinary items (`ibq`) relative to lagged quarterly total assets (`atq`), i.e.,

$$
ROA_q = \frac{IBQ_q}{ATQ_{q-1}}.
$$

Finally, the earnings announcement date must be after the fiscal quarter-end date.

ROE - Return on Equity

Following Hou *et al.* (2015), we measure return on equity as quarterly income before extraordinary items (`ibq`) relative to lagged book equity (BEQ)

$$ROE_q = \frac{IBQ_q}{BEQ_{q-1}}.$$

Quarterly book equity (BEQ) is computed as the (a) book equity of shareholders plus (b) balance sheet deferred taxes and investment tax credit minus (c) the book value of preferred stock. Regarding (a), we measure book equity of shareholders by shareholders' equity (`seqq`), or the sum of common equity (`ceqq`) and the par value of preferred stock (`pstkq`), or if all previous items are unavailable by total assets (`atq`) minus total liabilities (`ltq`). Regarding (b), the balance sheet deferred taxes and investment tax credit is `txditcq`, or `txdbq`, or zero if both are missing. Regarding (c), the book value of preferred stocks corresponds to `pstkrq`, or `pstkq`, or zero if both are unavailable. Finally, the earnings announcement date must be after the fiscal quarter-end date.

TBI - Taxable Income to Book Income

Following Lev and Nissim (2004), we measure taxable income to book income as pretax income (`pi`) scaled by net income (`ni`)

$$TBI_t = \frac{PI_t}{NI_t}.$$

We require positive pretax and net income.

A-1.6. Size

ME - Size

Following Fama and French (1992), we measure size as the natural logarithm of the market equity. We obtain market equity data from CRSP by multiplying the shares outstanding (`shrout` * 1,000, as they are given in thousands) with the corresponding share price (the absolute value of `altprc`), i.e.,

$$ME_t = \log\left(shrout_t * 1000 * |altprc_t|\right).$$

A-1.7. Trading frictions

AMI - Amihud Illiquidity Measure

Following Amihud (2002), we measure the illiquidity from daily return-to-volume (RV), which is the absolute daily return (r) scaled by the daily dollar trading volume (product of `vol` and `prc`), i.e.,

$$RV_t = \frac{|r_d|}{prc_t * vol_t}.$$

Then, the Amihud illiquidity measure corresponds to the average return-to-volume estimate over the last six months. We require at least 50 observations for this average and adjust the trading volume of NASDAQ stocks according to Gao and Ritter (2010).

BETA - Beta Relative to the Market

Following Fama and MacBeth (1973), we measure the market beta β_1 from monthly CRSP data. Specifically, we run the following time series regression over the previous five years:

$$r_t^e = \alpha + \beta_1 \cdot (\text{MKT}_t - R_t^f) + u_t.$$

Moreover, we require at least 24 monthly observations for the regression above.

DTV - Dollar Trading Volume

Following Brennan *et al.* (1998), we measure dollar trading volume from daily CRSP data as the average dollar trading volume (product of `vol` and `prc`) from month $t-6$ to month $t-1$ (with at least 50 observations). Moreover, we adjust dollar trading volume from NASDAQ according to Gao and Ritter (2010).

ISKEW - Idiosyncratic Skewness Relative to the Fama-French three-factor model

Following Bali *et al.* (2016), we regress daily excess returns in each month on the Fama-French three-factor model (with a minimum of 15 daily observations), i.e.,

$$r_t^e = \alpha + \beta_1 \cdot (\text{MKT}_t - R_t^f) + \beta_2 \cdot \text{SMB}_t + \beta_3 \cdot \text{HML}_t + u_t.$$

Then, idiosyncratic skewness is measured as the skewness of residuals u_t.

IVOL - Idiosyncratic Volatility Relative to the Fama-French model

Following Ang *et al.* (2006), we compute idiosyncratic volatility relative to the Fama-French three-factor model as the volatility of residuals from the following regression (with a minimum of 15 daily observations)

$$r_t^e = \alpha + \beta_1 \cdot (\text{MKT}_t - R_t^f) + \beta_2 \cdot \text{SMB}_t + \beta_3 \cdot \text{HML}_t + u_t.$$

MDR - Maximum Daily Return

Following Bali *et al.* (2011), we measure the maximum daily (excess) return in each month from daily CRSP data (with at least 15 return observations).

TUR - Share Turnover

Following Datar *et al.* (1998), we measure the daily share turnover of each stock as the number of shares traded (`vol`) scaled by the number of shares outstanding (`shrout`), i.e.,

$$T_t = \frac{vol}{shrout}.$$

Then, the share turnover for each firm in each month t is the average daily share turnover over the previous six months (with at least 50 observations). Moreover, we adjust the trading volume of NASDAQ stocks according to Gao and Ritter (2010).

A-1.8. Valuation

BM - Book Equity to Market Equity

Following Davis, Fama, and French (2000), we measure the book-to-market ratio as book equity (BE) from Compustat divided by market equity (ME) from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$BM_t = \frac{BE_t}{ME_t}.$$

The definition of book equity is (a) the book equity of shareholders plus (b) deferred taxes and investment tax credit (Compustat item `txditc` or `txdb` + `itcb` if `txditc` is unavailable) minus (c) the book value of preferred stock. Regarding (a), the book equity of shareholders is shareholders' equity (`seq`), or the sum of common equity (`ceq`) and the par value of preferred stock (`pstk`), or, if all previous items are unavailable, total assets (`at`) minus total liabilities (`lt`). Regarding (b), deferred taxes (`txditc`) or investment tax credits (`txdb` plus `itcb`) if deferred taxes are unavailable. Regarding (c), the book value of preferred stock corresponds in the following order either to the redemption value (`pstkrv`), or the liquidation value (`pstkl`), or if all previous items are unavailable to the par value (`pstk`). We replace the missing values in items (b) and (c) with zero.

CFM - Cash Flow to Market Equity

Following Lakonishok *et al.* (1994), we measure the cash-flow-to-price ratio as income before extraordinary items (`ib`) plus depreciation (`dp`) scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$CFM_t = \frac{IB_t + DP_t}{ME_t}.$$

We exclude all stocks with negative cash flows.

DM - Debt to Market Equity

Following Bhandari (1988), we measure the debt-to-market ratio as short-term debt (`dlc`) plus long-term debt (`dltt`) from Compustat divided by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$DM_t = \frac{DLC_t + DLTT_t}{ME_t}.$$

We exclude stocks with missing `dlc` and `dltt` observations.

EBM - Enterprise Book Equity to Market Equity

Following Penman *et al.* (2007), we measure enterprise book equity scaled by market equity as net debt plus book equity scaled by net debt plus market equity, i.e.,

$$EBM_t = \frac{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t + BE_t}{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t + ME_t},$$

where `dltt` corresponds to long-term debt, `dlc` to current debt, `pstk` to the value of preferred stock, `dvpa` to preferred dividends in arrears, `tstkp` to preferred treasury stock, `che` to cash and short-term investments, and ME to the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD). Lastly, in this case, book equity BE is computed as common equity (`ceq`) plus `tstkp` minus `dvpa`. Missing observations in `dvpa` and `tstkp` are set to zero. Additionally, we require that the sum of net debt and book equity, as well as the sum of net debt plus market equity, is positive.

EM - Earnings to Market Equity

Following Basu (1983), we measure the earnings-to-price ratio as income before extraordinary items (`ib`) from Compustat divided by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$EM_t = \frac{IB_t}{ME_t}.$$

We exclude firms with negative earnings.

NDM - Net Debt to Market Equity

Following Penman *et al.* (2007), we measure net debt to price as net debt from Compustat scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$NDM_t = \frac{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t}{ME_t},$$

where `dltt` corresponds to long-term debt, `dlc` to current debt, `pstk` to the value of preferred stock, `dvpa` to preferred dividends in arrears, `tstkp` to preferred treasury stock, and `che` to cash and short-term investments. Lastly, missing observations in `dvpa` and `tstkp` are set to zero.

NPY - Net Payout Yield

Following Boudoukh *et al.* (2007), we measure the net payout yield as net payouts from Compustat scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$NPY_t = [(DVC_t + PRSTKC_t + \Delta PSTKRV_t \cdot 1_{\Delta PSTKRV<0}) - (SSTK_t - \Delta PSTKRV_t \cdot 1_{\Delta PSTKRV>0})]/ME_t,$$

where `dvc` are dividends from common stock, `prstkc` is the purchase of common and preferred stock, `pstkrv` is the value of the net number of preferred stocks outstanding, and `sstk` reflects the sale of common and preferred stocks. $1_{\Delta PSTKRV<0}$ is a dummy variable that is one if the annual change in `pstkrv` is negative and zero otherwise. Moreover, we exclude stocks with negative net payouts.

OCM - Operating Cash Flow to Market Equity

Following Desai *et al.* (2004), we measure the ratio of operating cash flows to price as operating cash flows (OC) from Compustat divided by the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$OCM_t = \frac{OC_t}{ME_t}.$$

Before 1988, we measure OC as funds from operations (`fopt`) minus the change in working capital (item `wcap`). Thereafter, we measure OC as net cash flows from operating activities (`oancf`). Moreover, we exclude firms with negative operating cash flows.

SM - Sales to Market Equity

Following Barbee Jr *et al.* (1996), we measure the sales-to-price ratio (SM) as sales (`sale`) from Compustat divided by the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, `shrout`, times the absolute value of their price, `altprc`, in million USD), i.e.,

$$SM_t = \frac{SALE_t}{ME_t}.$$

Affiliation:

Christoph Frey
Management School
Lancaster University
Bailrigg
Lancaster LA1 4YX United Kingdom
E-mail: christoph.frey@gmail.com

Christoph Scheuch
School of Business and Economics
Humboldt University of Berlin
Dorotheenstraße 1
10099 Berlin Germany
E-mail: christoph.scheuch@hu-berlin.de

Stefan Voigt
Department of Economics
University of Copenhagen
Øster Farimagsgade 5
1353 Copenhagen K Denmark
E-mail: stefan.voigt@econ.ku.dk

Patrick Weiss
Department of Business and Economics
Reykjavík University
Menntavegur 1
102 Reykjavík Iceland
E-mail: patrickw@ru.is

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd